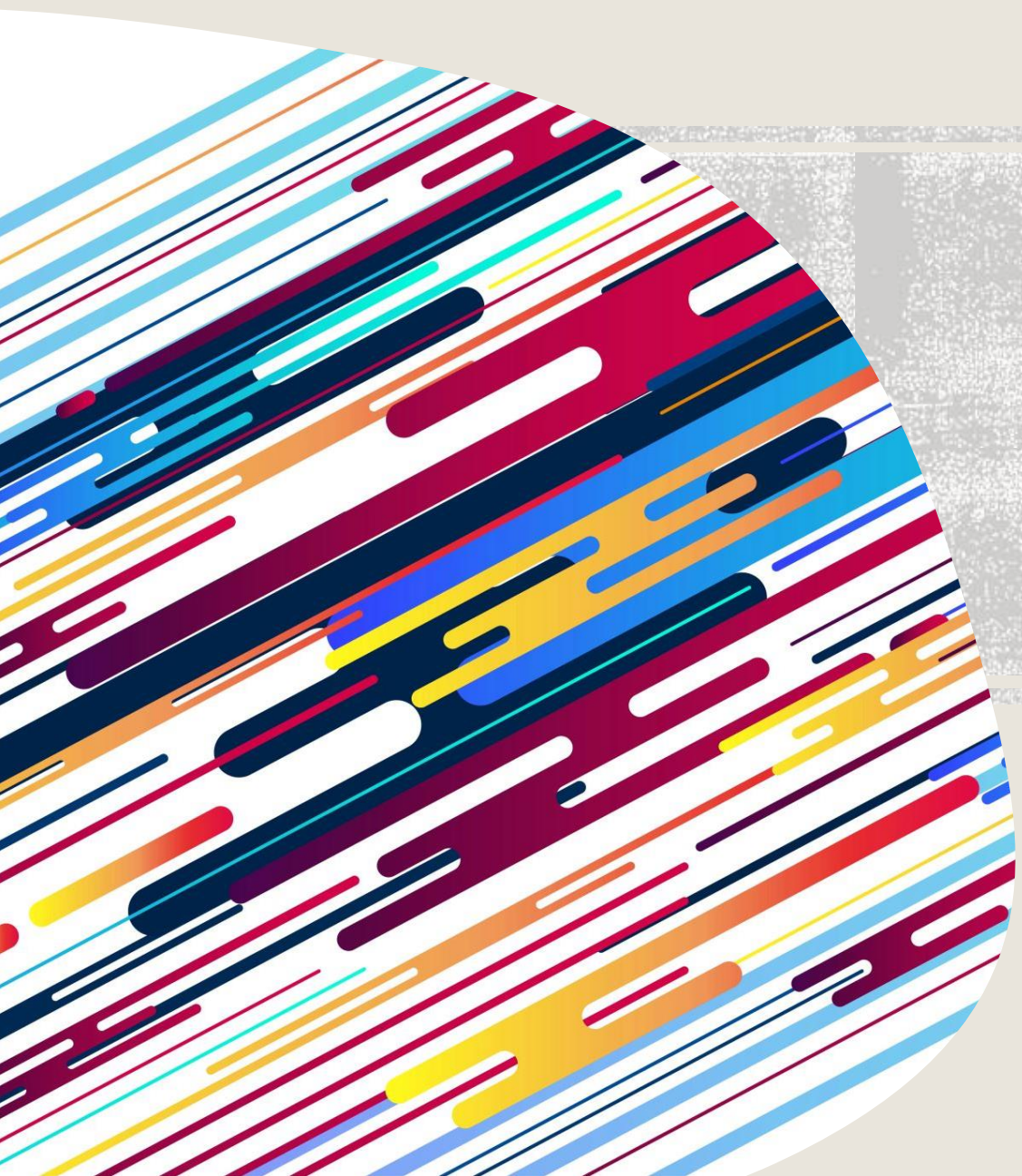




DOMAIN DRIVEN DESIGN (DDD)

1

LEARNING GOALS

By the end, students should be able to:

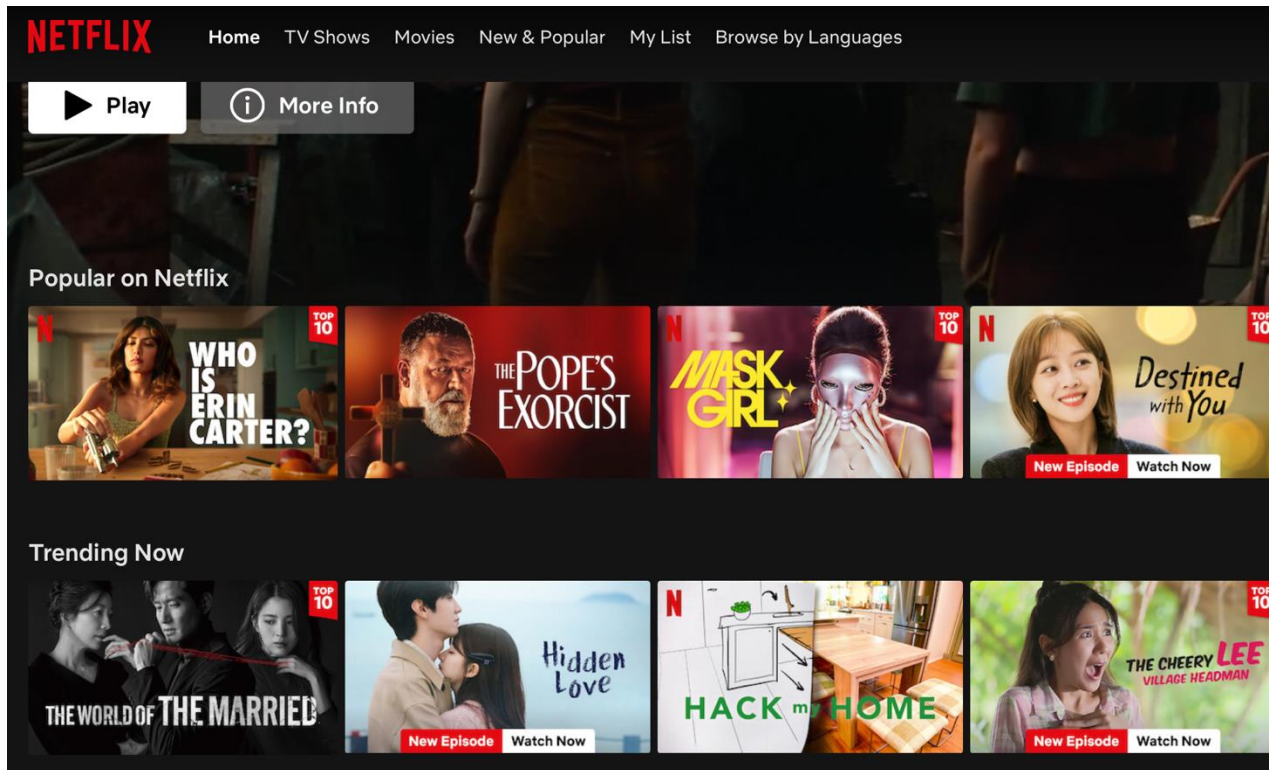
- Explain core **DDD concepts**, e.g., **ubiquitous languages** in their own words.
- Model a domain using **Entities**, **Value Objects**, and **Aggregates**.
- Understand how to define and work within **Bounded Contexts**.
- Apply DDD patterns to a simple project.

WHAT IS DDD?

DDD focuses on modelling the software that match the business's core domain
(business-tech alignment)

The software design will correspond to the input from domain experts

What is NETFLIX' core domain?



What is the core business of Netflix?

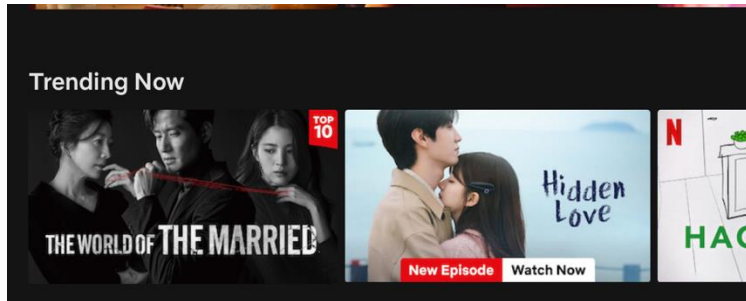
At the core of Netflix's business model is its subscription-based model, content production and acquisition, and data-driven personalization. Netflix, the world's leading streaming service, has revolutionized the way people consume entertainment. 27 n.w. 2567



Untaylored

<https://www.untaylored.com> › post › netflix-explaining-t...

Netflix - Explaining the Business Model and Revenue Streams



Netflix is a subscription-based streaming service that allows our members to watch TV shows and movies on an internet-connected device. Depending on your plan, you can also download TV shows and movies to your Android phone or tablet, iPhone, iPad, or Google Chromebook and watch without an internet connection.



Netflix Help Center

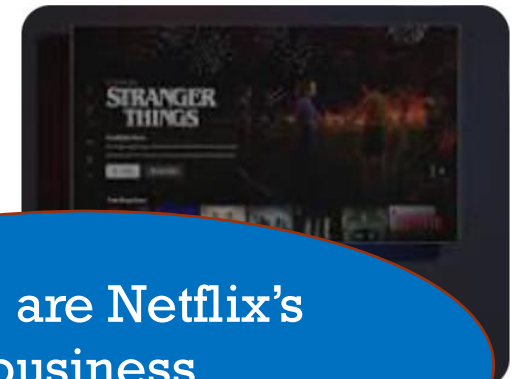
<https://help.netflix.com> › node

What is Netflix? - Netflix Help Center

What is NETFLIX' core domain?

?

What are Netflix's business capabilities?



SOFTWARE DEVELOPMENT PROCESS

- In order to create a good software, you have to know what that software is all about.
- You cannot create a banking software system unless you have a good understanding of what banking is all about.
- For example, design a software for banking, one must understand the **domain** of banking.
- Software needs to incorporate the **core concepts and elements of the domain**, and to precisely realize the relationships between them.
- Software which does not have its roots planted deeply into the domain will not react well to change over time.

ต้องเข้าใจระบบนั้น ๆ
ยิ่งเข้าใจมาก ก็ยิ่งออกแบบได้ดีและต่อ ยอดได้ง่าย



How about designing a software for
figth-control system?



What do we need to know?

Develop ideas and
try to extract knowledge
from domain experts

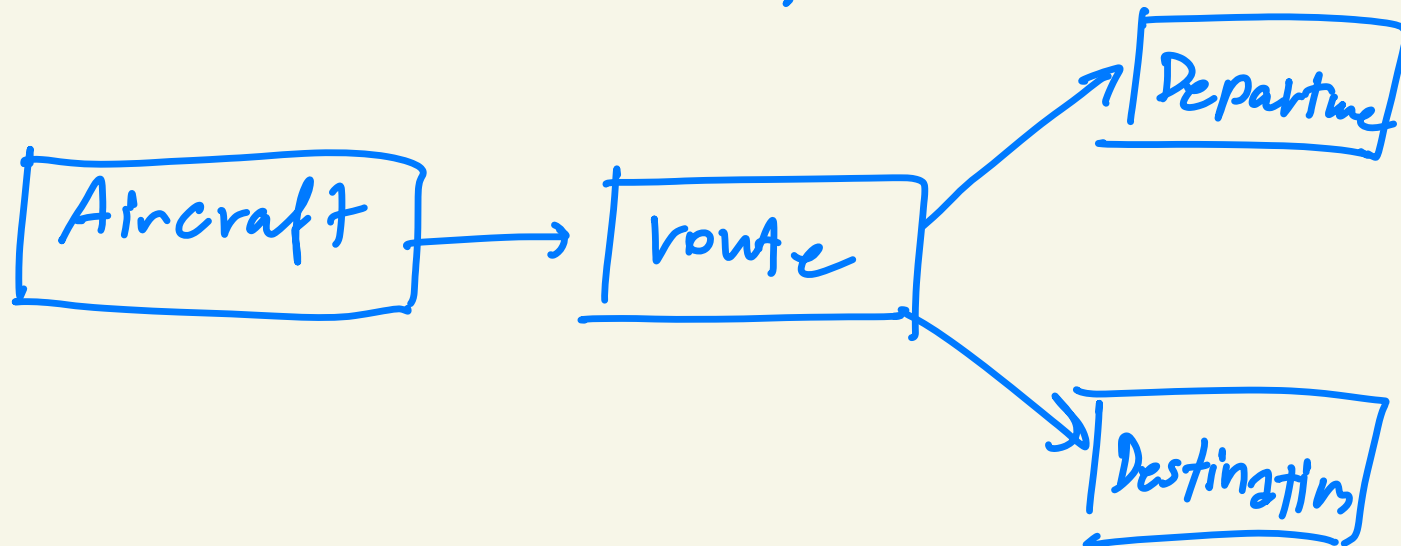
An airplane flight control system

Vocables: taking off, landing, mid-air,
danger of collision, etc....



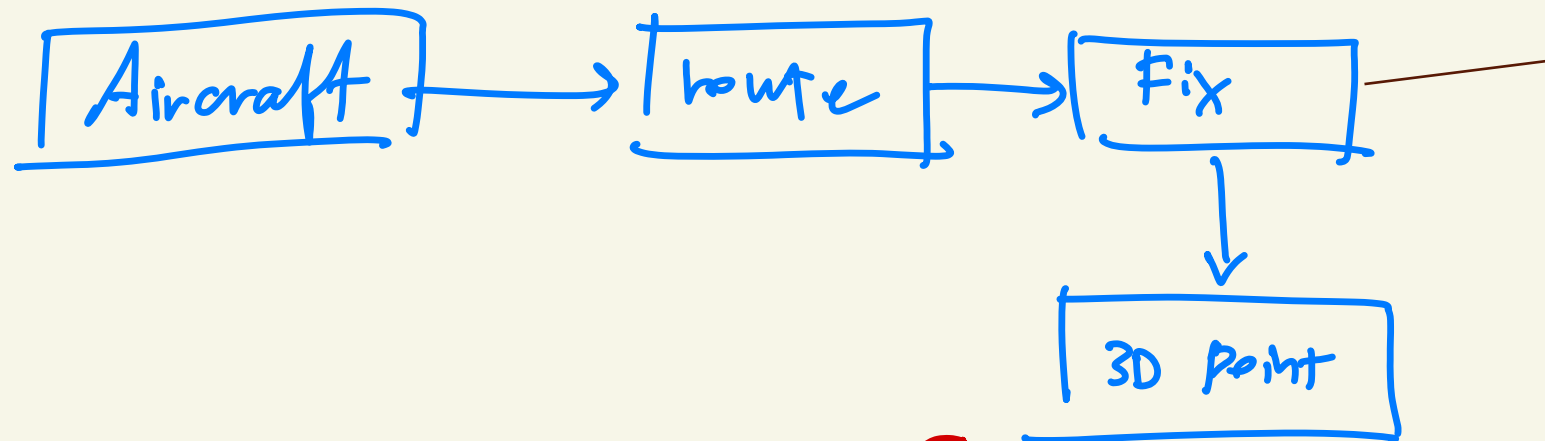
the controller says each plane is
assigned a flight plan....

!! a route !!



Develop ideas and
try to extract knowledge
from domain experts

the route is made up of small segments, which put together constitute some sort of crooked line from departure to destination.



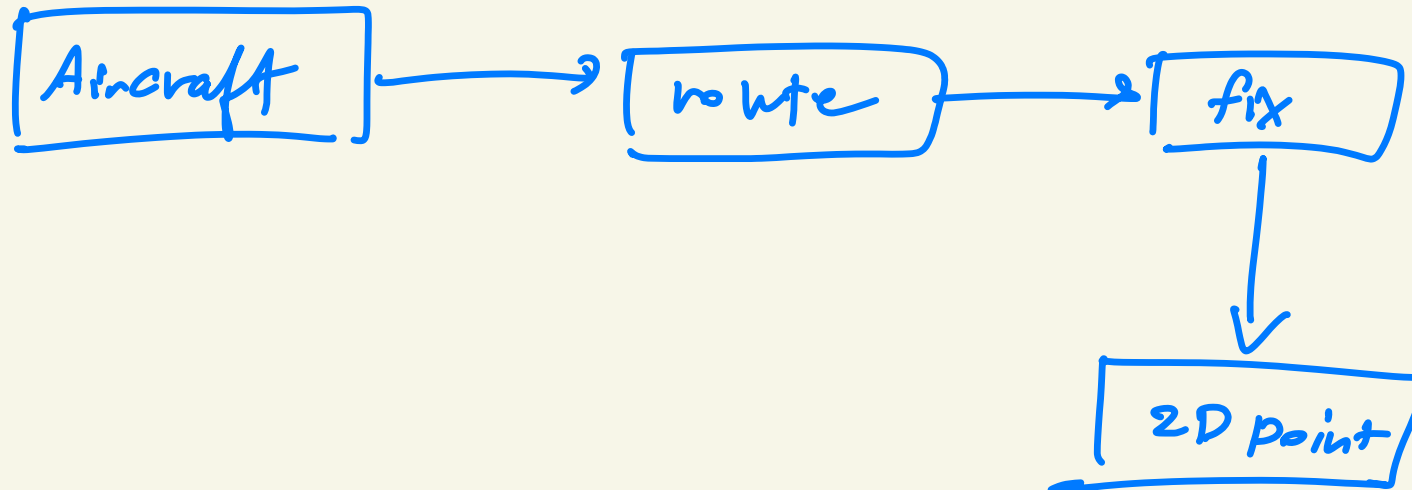
our view of plane in the air

Develop ideas and try to extract knowledge from domain experts

A precise point in space defined by coordinates,

Develop ideas and
try to extract knowledge
from domain experts

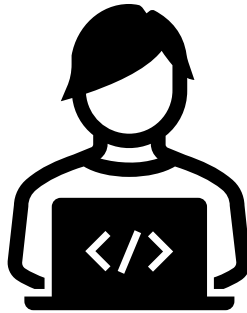
So, revise the diagram ...



Controllers see points as projection
on earth ...

Ubiquitous languages

The need for a common language



Classes, methods, algorithms,
full stack, front-end, backend,
Patterns, SQL, NoSQL, cloud,
Token,



Planes, routes, altitudes, latitude,
longitude, deviances from the normal
route, plane trajectories....

UBIQUITOUS LANGUAGE CONCEPTS

- A core principle of **domain-driven design** is to **use a language based on the model**.
- Language **does not appear overnight**.
- We need to find key concepts which define the domain. Design and find corresponding words for them and start using them consistently.

UBIQUITOUS LANGUAGE IN FLIGHT CONTROL SYSTEM

What is the ubiquitous language in insurance?

Deductible,
High deductible health plan,
Health saving account,
Premium,
Copayment,
Coinsurance,
Out-of-pocket maximum,
HMO (Health Maintenance Organization),
POS (Point of service),
PPO (preferred provider organization)



UBIQUITOUS LANGUAGE IN FLIGHT CONTROL SYSTEM

What is the ubiquitous language your term project?



What is the ubiquitous language of Netflix?

What is the core business model of Netflix?

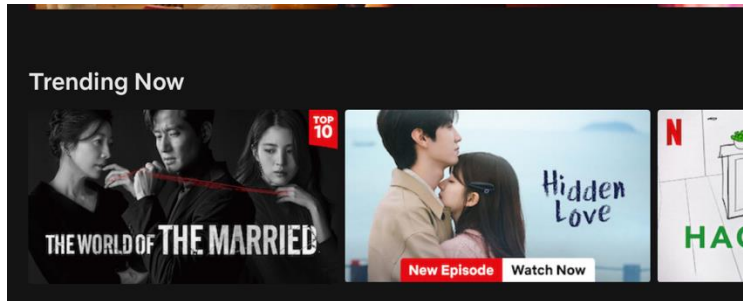
At the core of Netflix's business model is a subscription-based model, content production and acquisition, and data-driven personalization. Netflix, the world's leading streaming service, has revolutionized the way people consume entertainment. 27 n.w. 2567



Untaylored

<https://www.untaylored.com> › post › netflix-explaining-t...

Netflix - Explaining the Business Model and Revenue Streams



Netflix is a subscription-based streaming service that allows our members to watch TV shows and movies on an internet-connected device. Depending on your plan, you can also download TV shows and movies to your Android phone or tablet, iPhone, iPad, or Google Chromebook device and watch without an internet connection.



Netflix Help Center

<https://help.netflix.com> › node

What is Netflix? - Netflix Help Center

What is NETFLIX' core domain?

?

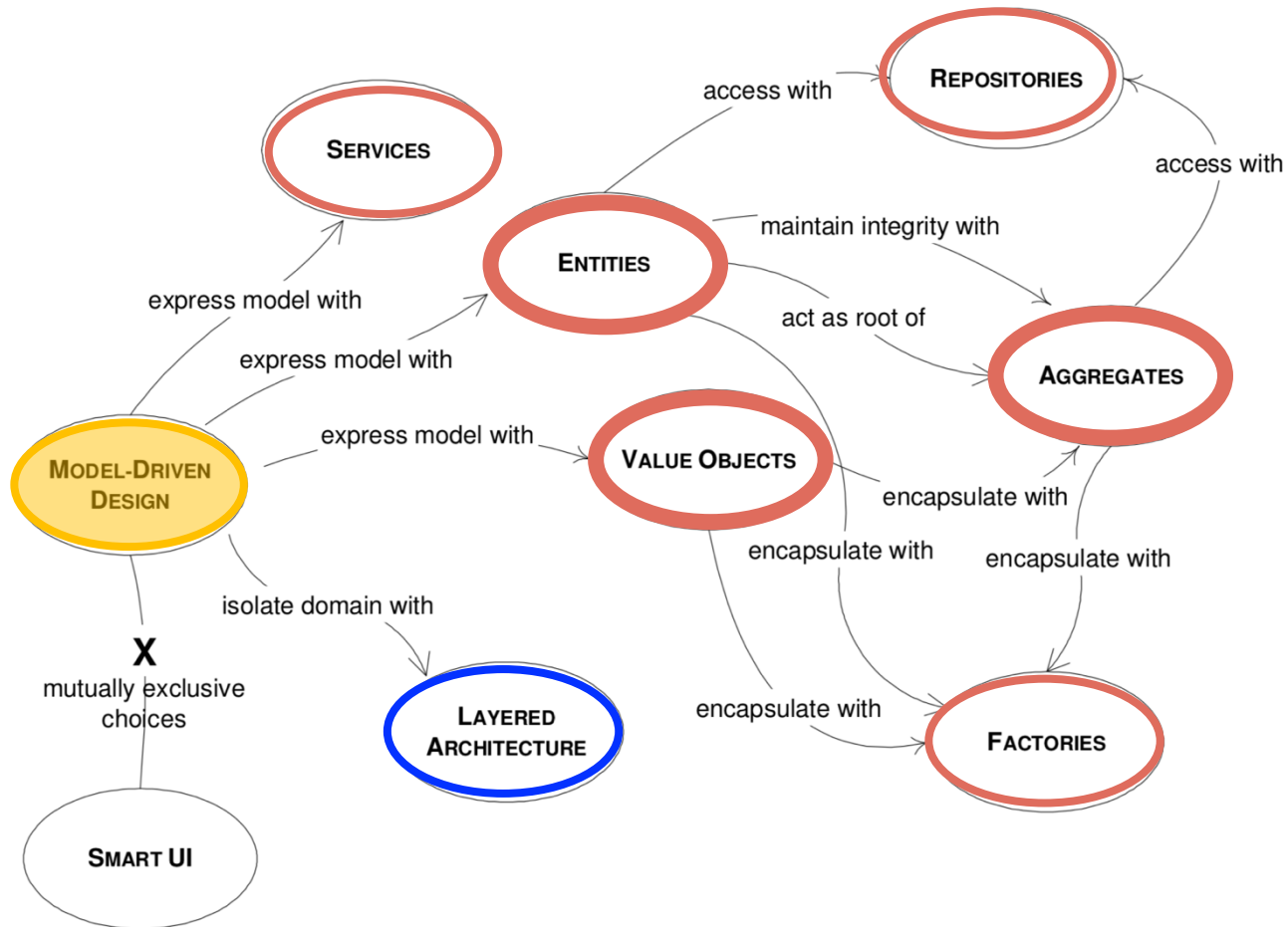


Model-Driven Design

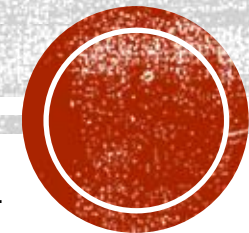
The goal of domain-driven design is to create better software by focusing on a model of the domain rather than the technology.

Avram A., Marinescu F., Domain-Driven Design Quickly: A summary of Eric Evans' Domain Driven Design
<https://www.lulu.com/content/325231?page=1&pageSize=4>

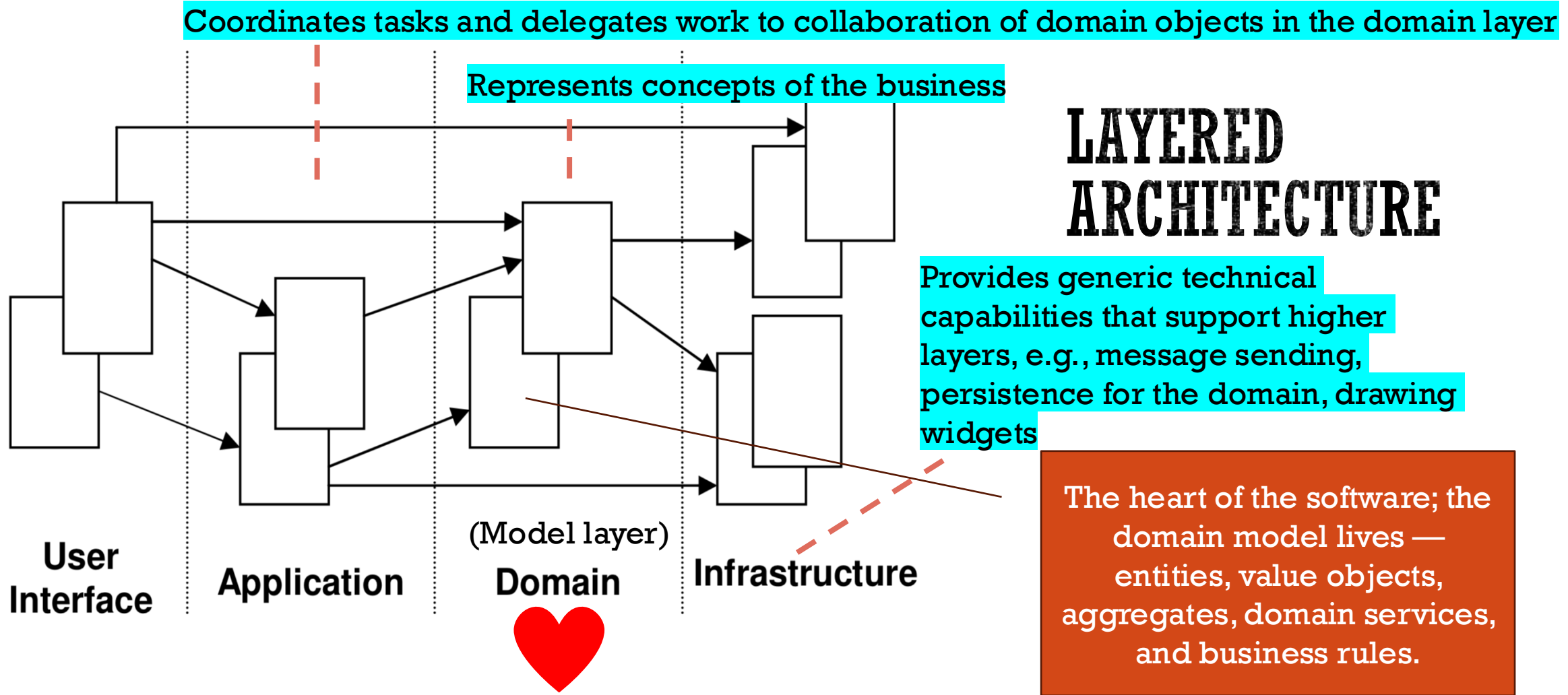
Eric Evans. Domain Driven Design. 2003



THE BUILDING BLOCKS OF A MODEL-DRIVEN DESIGN



A common architectural solution for domain-driven designs contain **four** conceptual layers:



Avram A., Marinescu F., Domain-Driven Design Quickly: A summary of Eric Evans' Domain Driven Design

<https://www.lulu.com/content/325231?page=1&pageSize=4>

ENTITIES

- An **Entity** is an object defined primarily by **its identity**, not by its attributes. Two entities with identical attribute values are still different objects if their identities differ.
- That identity **persists across the entire lifecycle** of the object — even as its attributes change over time, it remains "the same thing." e.g., Plane is still the same plane after a repaint, an engine swap, or a change in route
- Because **identity — not current state** — is what matters, entities are usually given an explicit, stable identifier (e.g., a plane's tail number, a flight plan's flight number) so the system can track and distinguish them reliably, even across restarts, database saves, and updates.
- Examples: Plane, Flight plan, Route, Fix, Airport

What are entities in
RegChula system?

What are entities in a
match maker system?

VALUE OBJECTS

- A **Value Object** describes some aspect or characteristic of the domain, and — unlike an Entity — it **has no identity of its own**.
- Two value objects with the same attributes are considered **interchangeable and equal**. For example, two Money objects both representing "\$50 USD" are simply *the same value*.
- **Rule of thumb for modeling:** default to Value Objects. Only promote an object to an Entity if it truly needs a persistent identity that must be tracked across state changes.
- **Value Objects should be immutable.** Rather than modifying a Value Object's internal state, you create a *new* instance with the desired values and replace the old one.

VALUE OBJECTS (EXAMPLE)

- **Examples (Air travel booking system):** The **flight code** (e.g., "AA123") is a **Value Object** — it has no identity of its own and can be safely *shared* across many customers who book the same flight.
- A customer's **booking**, by contrast, *is not a Value Object* — *it's an Entity*, since it gets its own booking reference that persists across changes (seat reassignment, cancellation, rebooking), even if two bookings share the same flight code.
- **Contrast with Entities:** where an **Airport, Plane, or Booking** needs a stable identity tracked over time (an Entity), a **FlightCode** or **Fare** is just a descriptive value attached to that entity — replaceable and comparison-by-value only.

IS "ADDRESS" A VALUE OBJECT?

It depends on context.

There's no universal answer — the same real-world concept (Address) can be modeled differently depending on what the software actually needs to do with it. This is a direct consequence of the **Ubiquitous Language** being tied to a specific **bounded context**.

IS "ADDRESS" A VALUE OBJECT?

Mail-order company → Address is a **Value Object**

- The address is only needed to confirm the credit card and route the parcel.
- It doesn't matter *whose* address it is or whether two customers share the same location (e.g., roommates).
- Since identity is irrelevant and only the attribute values (street, city, zip) matter, it's modeled as a Value Object.

IS "ADDRESS" A VALUE OBJECT?

Postal service → Address is an **Entity**

- The system organizes delivery routes using a hierarchy: regions → cities → postal zones → blocks → individual addresses.
- Each address derives its zip code from its position in that hierarchy.
- If the postal service reorganizes postal zones, an address needs to **persist its identity** and "come along for the ride" as its attributes (like zip code) change.
- Since the address must be tracked and updated over time while remaining "the same address," it qualifies as an Entity.

IS "ADDRESS" A VALUE OBJECT?

- **Electric utility company** → Address is an **Entity** (with an alternative modeling option)
- **An address** represents **a specific service destination** — the company needs to know exactly which physical location is receiving power, even if multiple customers (e.g., roommates) share that address.
- **Because the company must track and distinguish that specific service destination over time, Address becomes an Entity.**

IS "ADDRESS" A VALUE OBJECT?

- **Alternative model:** instead of making Address an Entity, the company could introduce a new Entity called Dwelling (the actual place receiving service) and attach Address to it merely as a descriptive attribute. In that case, Address reverts to being a Value Object, and Dwelling — not Address — carries the identity.

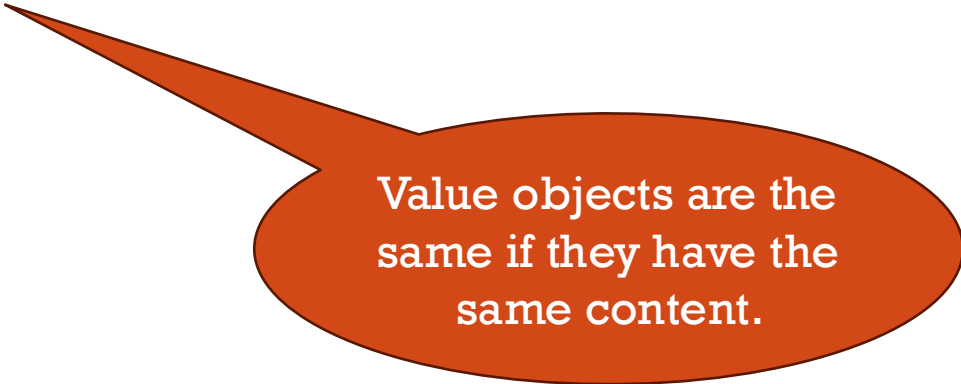
Don't ask "is X inherently an Entity or a Value Object?" — ask "**Does X need a tracked identity *for this specific model*, or is it fully described by its attributes?**"

The right answer depends on what the system cares about: **if identity and continuity across changes matter, it's an Entity**; **if only the current attribute values matter and interchangeability is fine, it's a Value Object.**

This also shows that **modeling is a design choice**, not a fixed classification — the same concept can (and should) be modeled differently in different bounded contexts.

MORE EXAMPLES OF VALUE OBJECTS

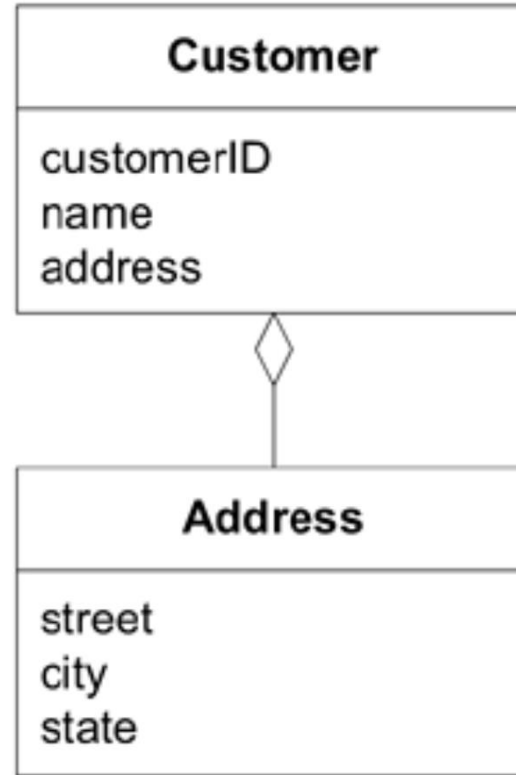
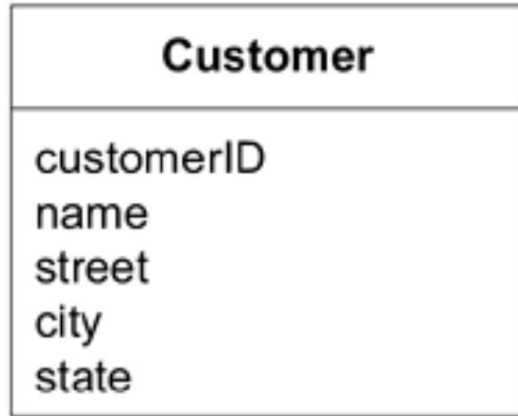
- Money: An amount and a currency (e.g., Thai Baht, US dollar)
- Date range: A start date and end date
- Color: RGB values or a named color.



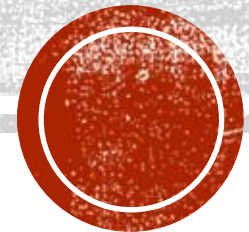
Value objects are the same if they have the same content.



รายวิชาบังคับที่ต้องเรียนในชั้นปี
3 เทอม 1



**ENTITY CAN
CONTAIN
VALUE
OBJECTS**



AGGREGATES

- An **Aggregate** is a cluster of associated objects (Entities and Value Objects) that are **treated as a single unit for the purpose of data changes**.
- Any update to the aggregate's data is made through the aggregate as a whole, ensuring the group stays internally consistent.
- The Aggregate is demarcated by a boundary which **separates the objects inside from outside**.
- Each Aggregate **has one root, an Entity**, the only accessible object from outside.
- **Why this matters:** the root is responsible for enforcing the aggregate's **invariants**

AGGREGATES EXAMPLE (AIR TRAVEL BOOKING SYSTEM)

- **A Booking** (Entity, Aggregate Root) might contain a list of Passenger Entities and several SeatAssignment Value Objects.
- External code (e.g., a payment service) never directly modifies a SeatAssignment — it must go through the Booking root, e.g., `booking.reassignSeat(passenger, newSeat)`.
- This lets Booking enforce rules like "a seat cannot be assigned to two passengers at once" or "total passengers cannot exceed aircraft capacity" — invariants that would be easy to violate if outside code could modify SeatAssignment objects directly.

Root ENTITY has
global identity

Root ENTITY is
responsible to check
invariants

A change within the
AGGREGATE boundary
need to satisfy invariants

EXAMPLE OF AN AGGREGATE

The two invariants can be enforced
in the "rotate" method—currently
the only external access to change
tire position.

A delete operation must
remove everything within
the AGGREGATE boundary

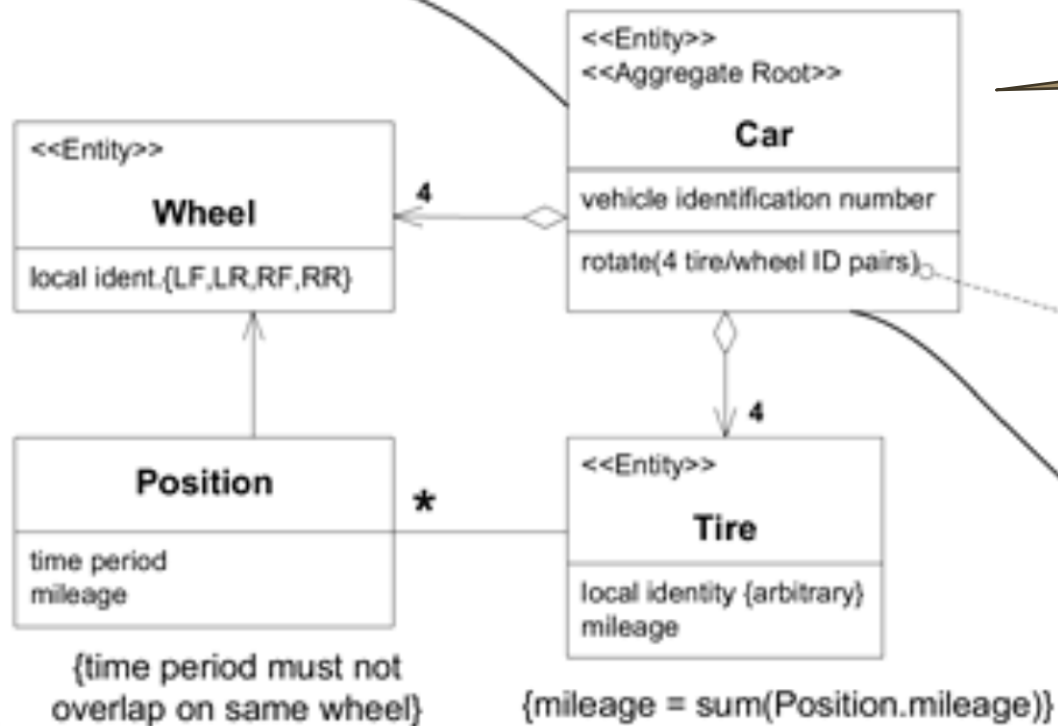


Figure 6.3. AGGREGATE invariants

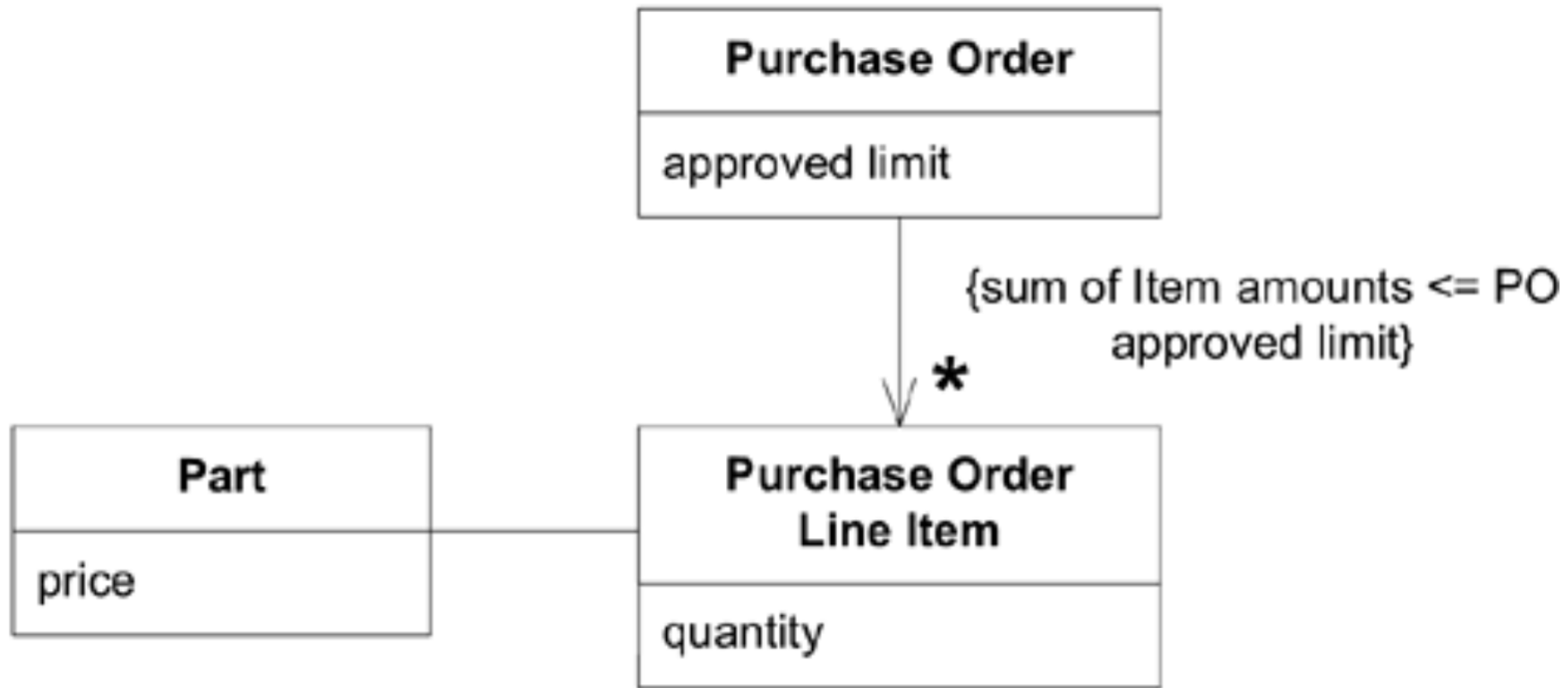
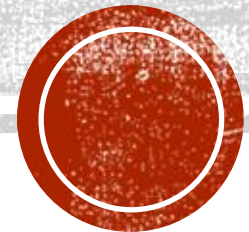


Figure 6.4. A model for a purchase order system

**EXAMPLE
OF AN
AGGREGATE**



AGGREGATES

PO #0012946 Approved Limit: \$1,000.00				
Item #	Quantity	Part	Price	Amount
001	3	Guitars	@ 100.00	300.00
002	2	Trombones	@ 200.00	400.00
Total:				700.00

Figure 6.5. The initial condition of the PO stored in the database

Original PO

These individual edits
cause no errors

Each item edited simultaneously

George adds guitars in his view

PO #0012946 Approved Limit: \$1000.00				
Item #	Quantity	Part	Price	Amount
001	5	Guitars	@ 100.00	500.00
002	2	Trombones	@ 200.00	400.00
Total:				900.00

Amanda adds a trombone in her view

PO #0012946 Approved Limit: \$1,000.00				
Item #	Quantity	Part	Price	Amount
001	3	Guitars	@ 100.00	300.00
002	3	Trombones	@ 200.00	600.00
Total:				900.00

Figure 6.6. Simultaneous edits in distinct transactions

AGGREGATES

PO #0012946 Approved Limit: \$1,000.00				
Item #	Quantity	Part	Price	Amount
001	5	Guitars	@ 100.00	500.00
002	3	Trombones	@ 200.00	600.00
Total:				1,100.00

How can we solve this?

The merged of these edits however violates the limit!!!.

Figure 6.7. The resulting PO violates the approval limit (broken invariant).

AGGREGATES

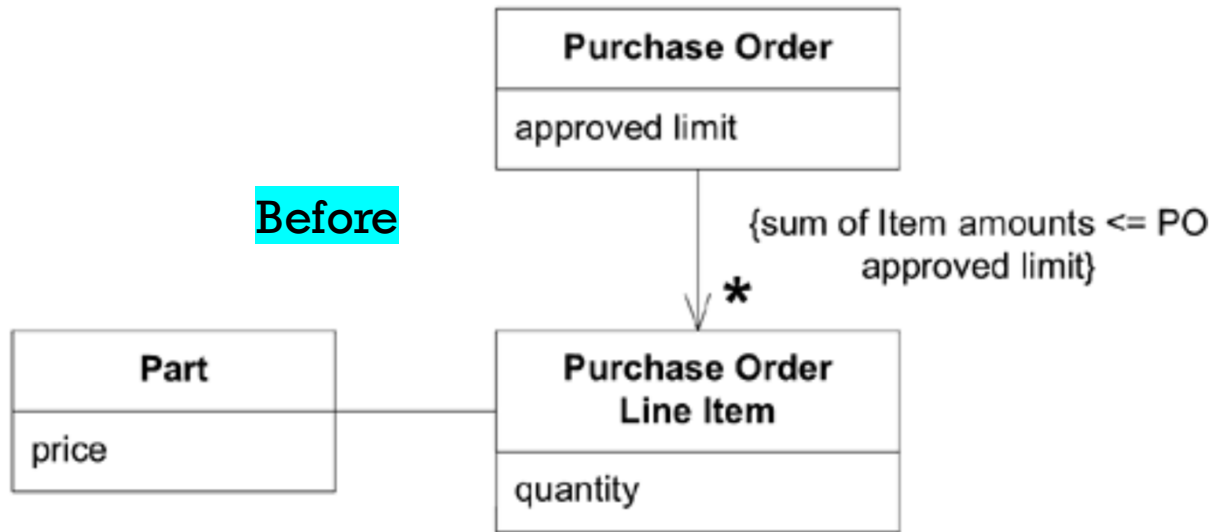


Figure 6.4. A model for a purchase order system

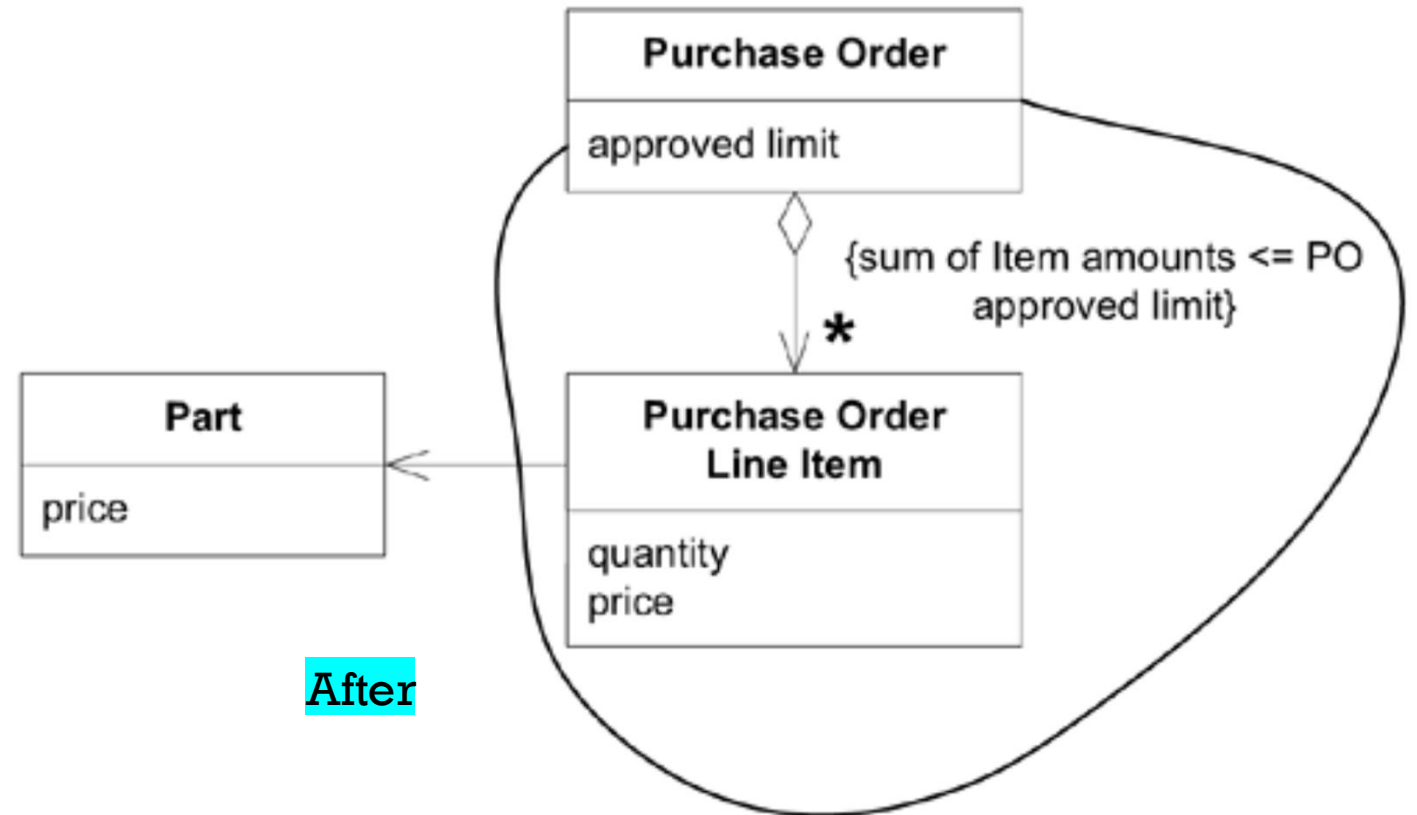


Figure 6.11. Price is copied into Line Item. AGGREGATE invariant can now be enforced.

REPOSITORIES

“Exposure of technical infrastructure and database access mechanisms complicates the client and obscures the MODEL-DRIVEN DESIGN.”

Repository pattern is a simple conceptual framework to capsule those solutions and bring back our model focus...

REPOSITORIES

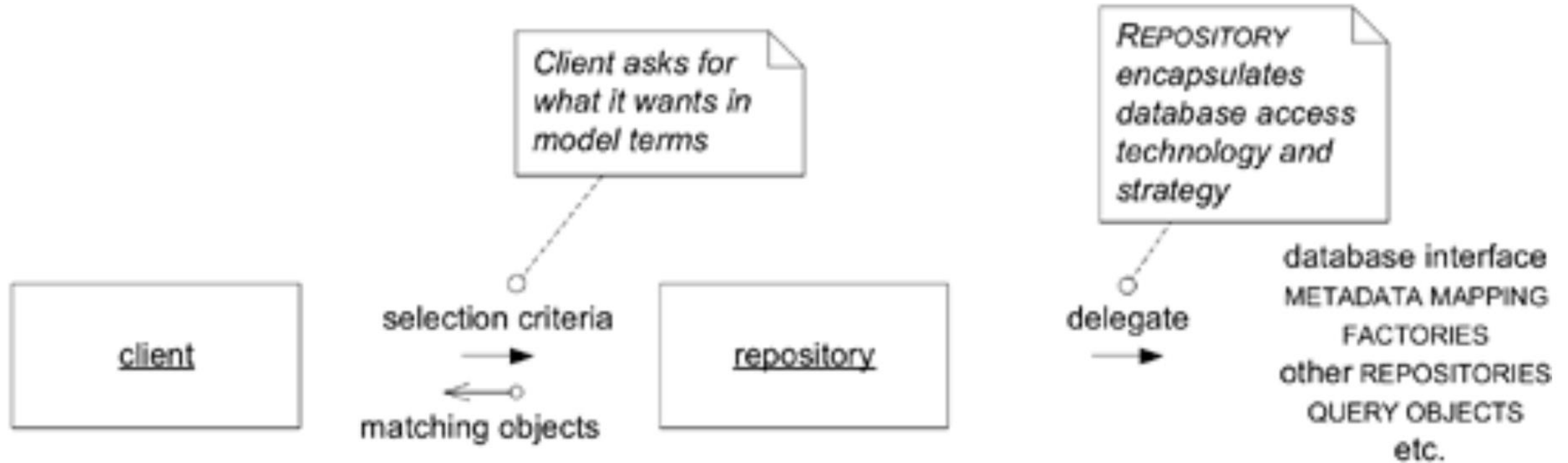


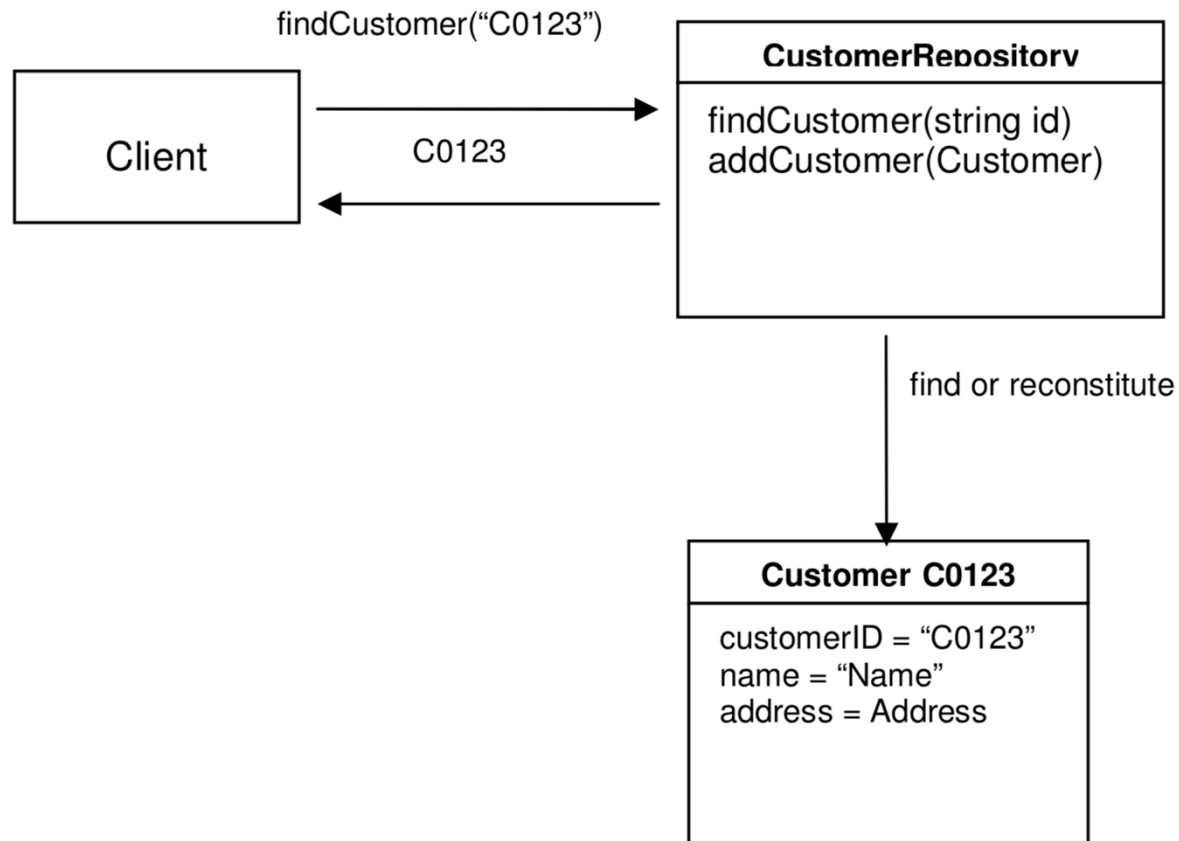
Figure 6.18. A REPOSITORY doing a search for a client

REPOSITORIES ADVANTAGES

- They **present clients with a simple model** for obtaining persistent objects and maintaining their life cycle.
- They **decouple application and domain design from persistence technology**, multiple database strategies, or even multiple data sources.
- They **communicate design decisions** about object access.
- They allow **easy substitutions** of a dummy implementation, for use in testing.

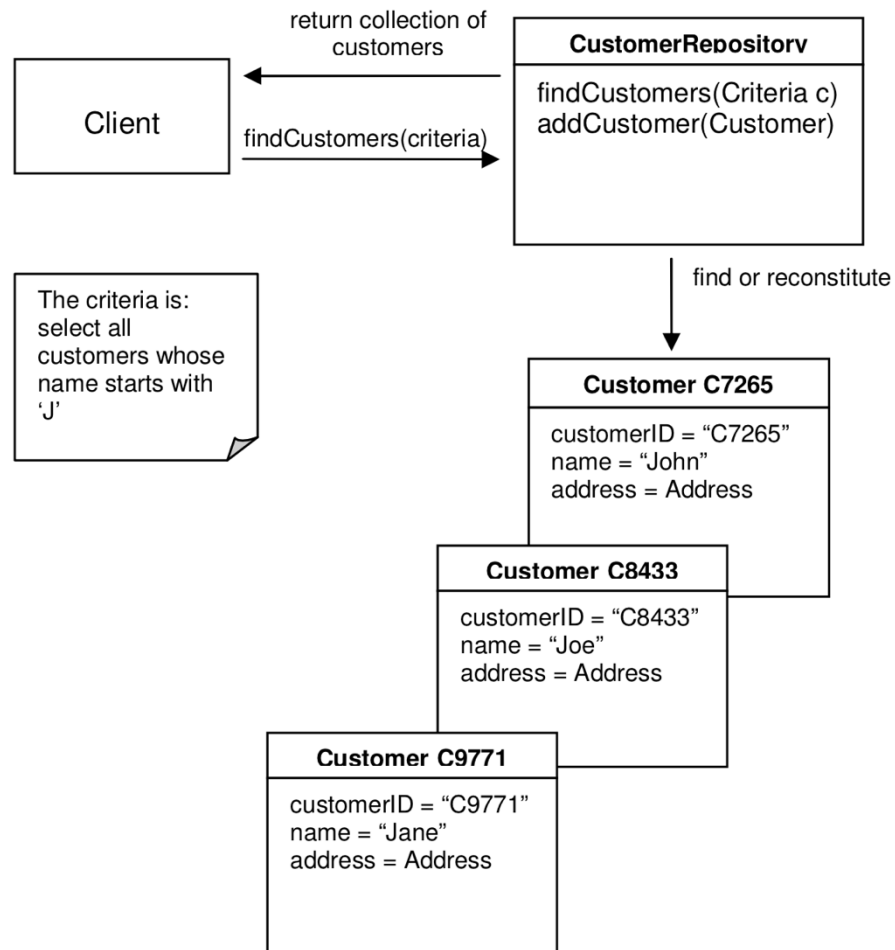
REPOSITORIES EXAMPLE (AIR TRAVEL BOOKING SYSTEM)

- **Example (Air travel booking system):**
BookingRepository.findByConfirmationCode("AA123XY") — retrieves a fully-formed **Booking aggregate** without the caller needing to know about joins, tables, or foreign keys.
- In tests, an InMemoryBookingRepository can stand in for the real database-backed one, letting you test booking logic without spinning up an actual database.



REPOSITORIES

A repository encapsulates all the logic needed to obtain object references.



REPOSITORIES

A repository encapsulates all the logic needed to obtain object references.

SERVICES

- Service is used to represent **behavior** within a domain.
- It can **group related functionalities** which serves the Entities and Value objects.
- It is much better to declare service explicitly. Hence, make the operation itself easy to find, name, and reason about in the Ubiquitous Language.
- Services **act as interfaces that exposes operations**, which should be **stateless**.
- Services **provide a point of connection for many objects**.

SERVICES (AIR TRAVEL BOOKING SYSTEM)

- `RebookingService.rebook(booking, newFlight)` — coordinates checking seat availability, canceling the old seat, applying rebooking fees, and updating the Booking aggregate. This clearly doesn't belong on Flight or Booking alone, since it spans both.
- `FarePricingService.calculateFare(route, date, seatClass)` — computes a price using several inputs, with no natural "owner" object among Route, Plane, or Fare.

FACTORIES

- Factories are used to **encapsulate the knowledge necessary for object creation**, and they are especially useful to create Aggregates.
- They guarantee that an object (especially an Aggregate) is created in a valid, consistent state from the very start.
- They separate "how to build it" from "how to use it."

FACTORIES (AIR TRAVEL BOOKING SYSTEM)

- `BookingFactory.createBooking(customer, flight, seatPreference)` — checks seat availability, generates a confirmation code, initializes the booking status, and assembles the full Booking aggregate (with its Passenger and SeatAssignment parts) as one atomic, valid unit — rather than leaving client code to piece it together step by step and risk creating an invalid booking.

FACTORY METHOD on the root of the AGGREGATE.

Add elements inside a preexisting AGGREGATE

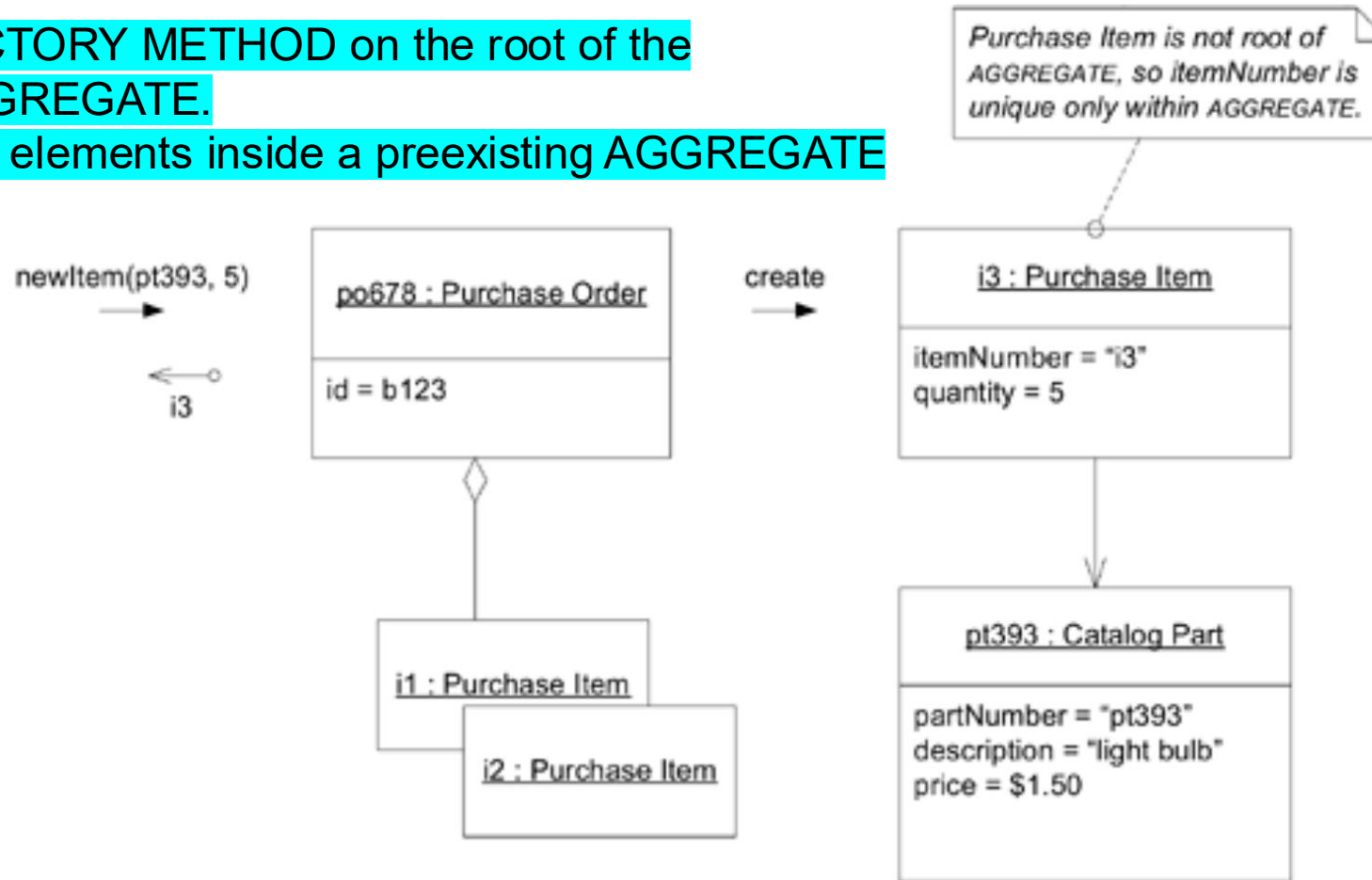


Figure 6.13. A FACTORY METHOD encapsulates expansion of an AGGREGATE.

FACTORIES AND THEIR SITES

Allow an AGGREGATE (Brokerage account) to create TradeOrders) that is not part of the AGGREGATE

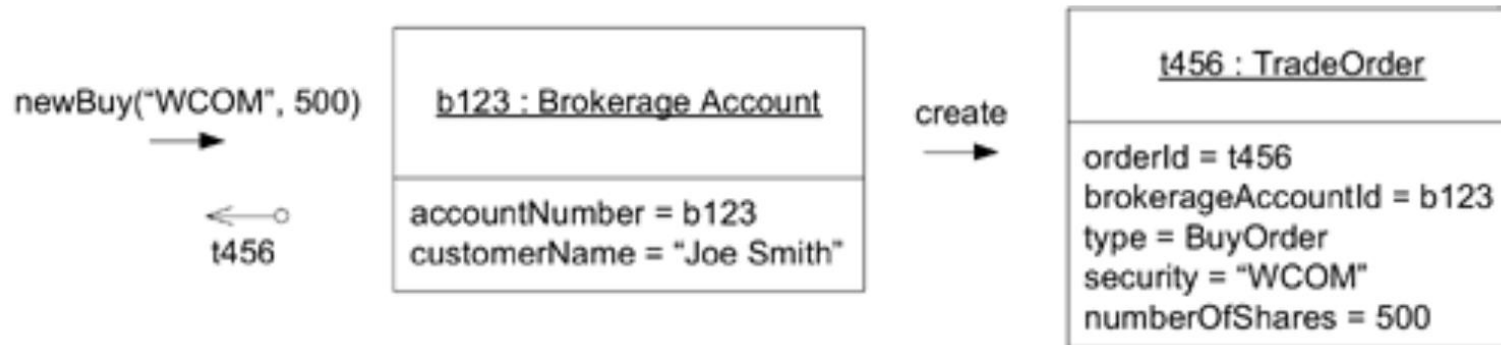


Figure 6.14. A FACTORY METHOD spawns an ENTITY that is not part of the same AGGREGATE.

FACTORIES AND THEIR SITES

A standalone FACTORY to build AGGREGATE

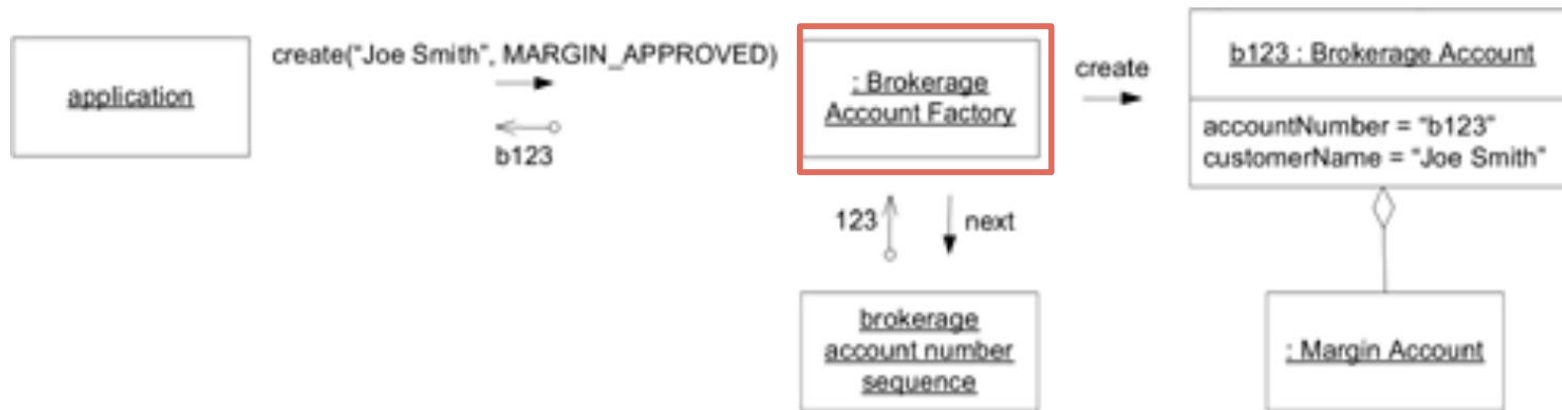
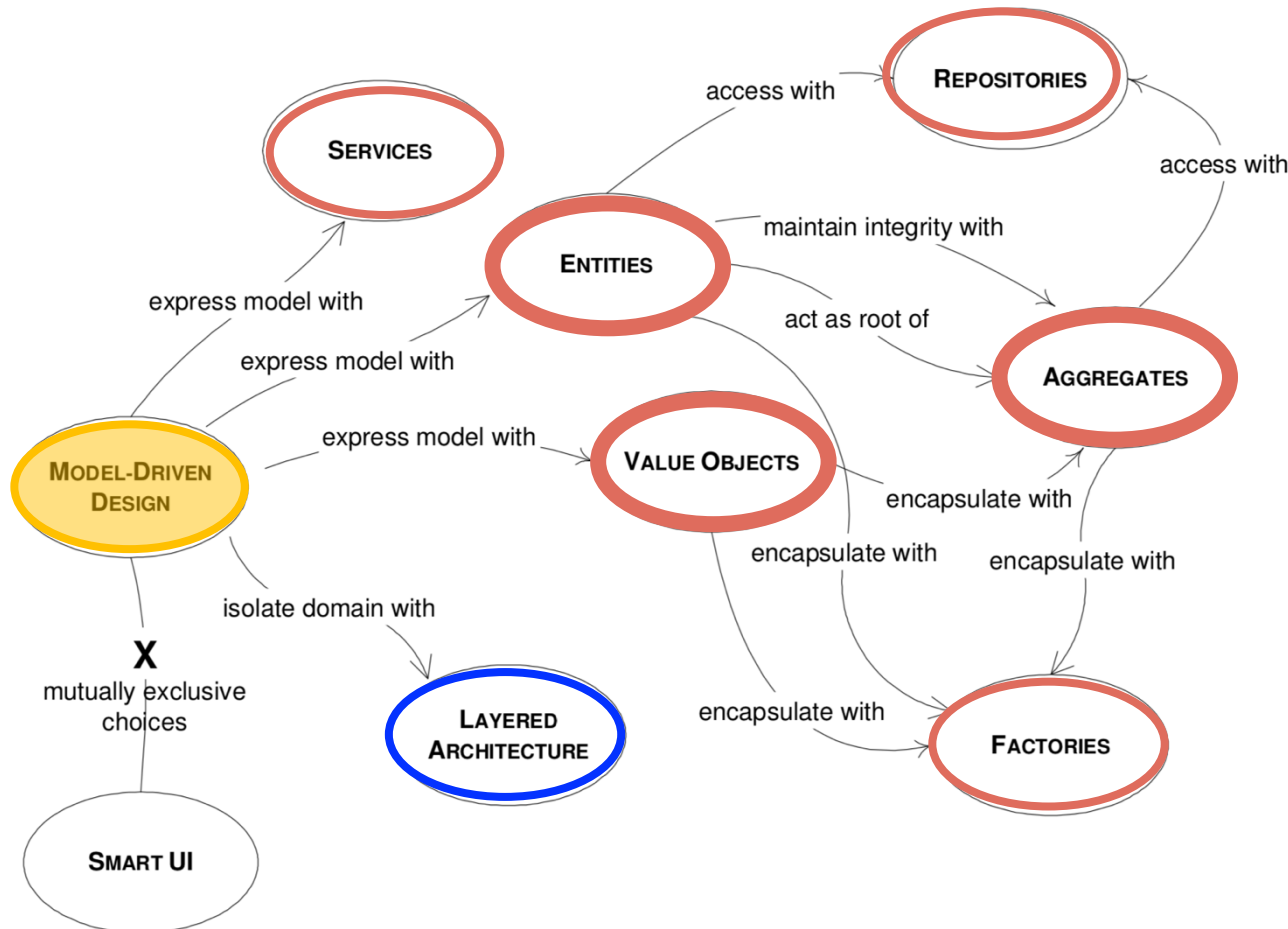


Figure 6.15. A standalone FACTORY builds AGGREGATE.

FACTORIES AND THEIR SITES

THE BUILDING BLOCKS OF A MODEL- DRIVEN DESIGN



Avram A., Marinescu F., Domain-Driven Design Quickly: A summary of Eric Evans' Domain Driven Design
<https://www.lulu.com/content/325231?page=1&pageSize=4>

DOMAIN EVENTS (INTRODUCED SINCE 2004)

- A domain event is representation of something that happened in the domain.
- Something happened that domain experts care about.
- Each event is represented as a domain object.
- Examples: *OrderPlaced*, *BookAddedToCatalog*, *PaymentReceived*.

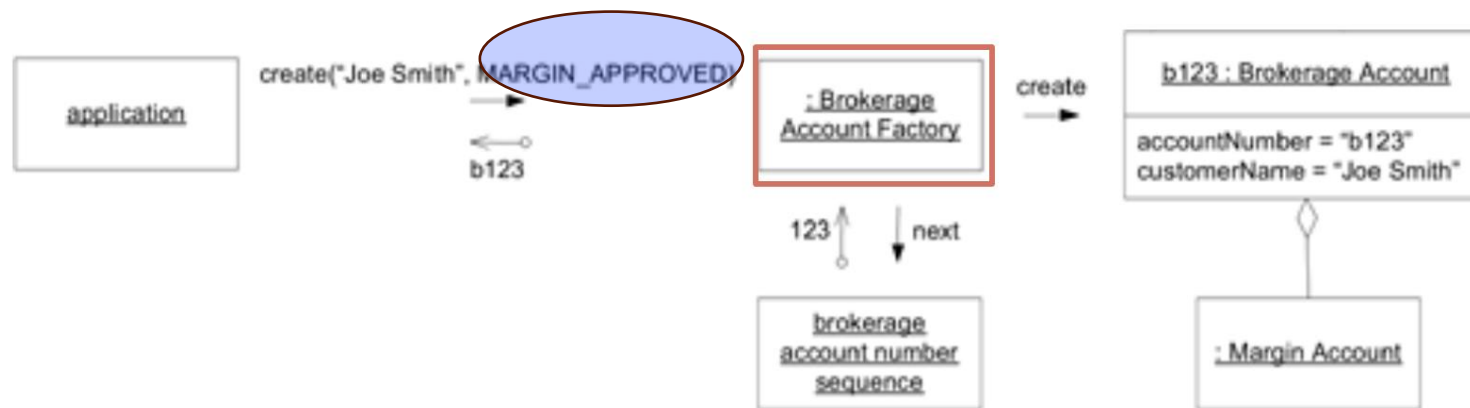
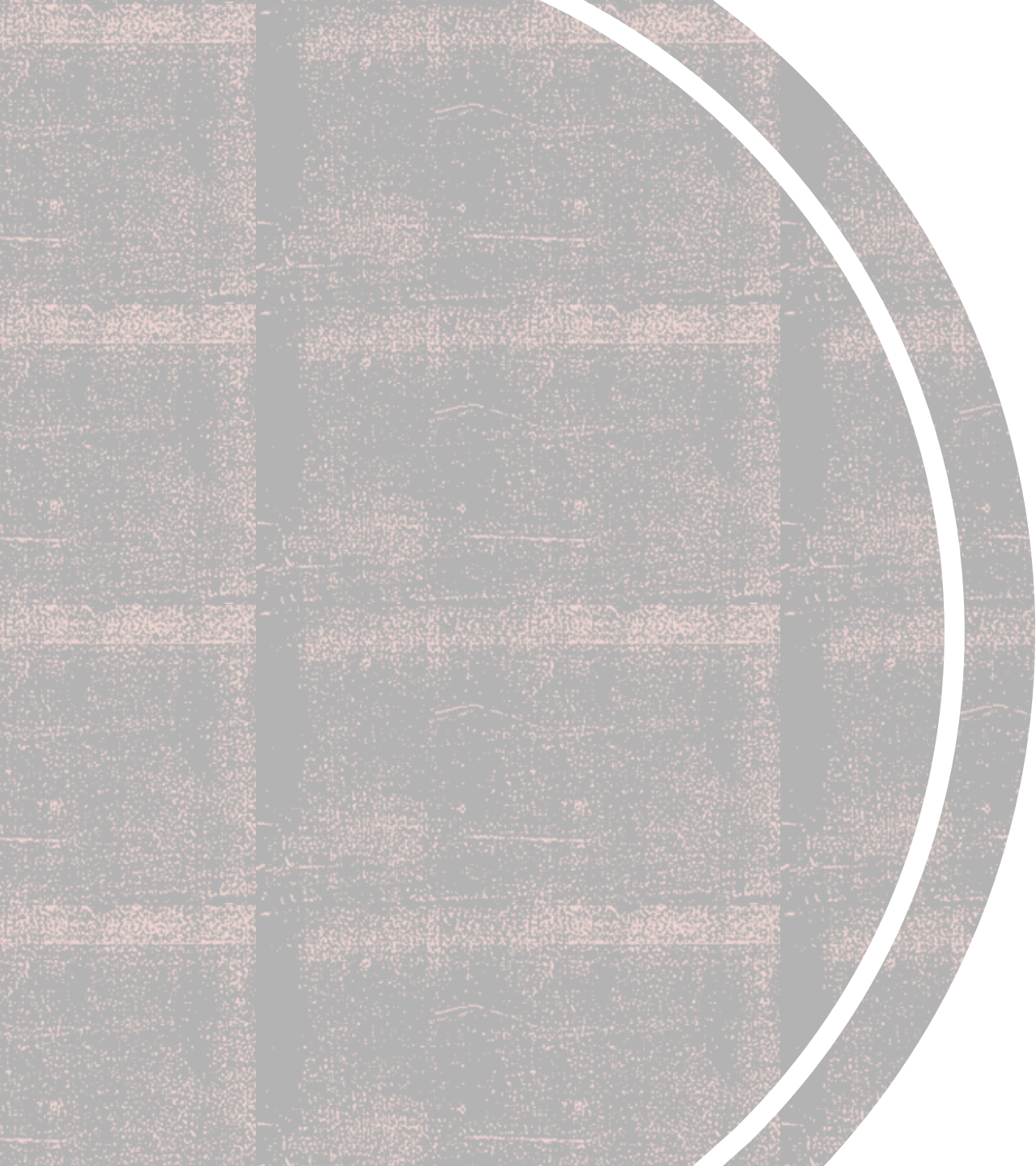


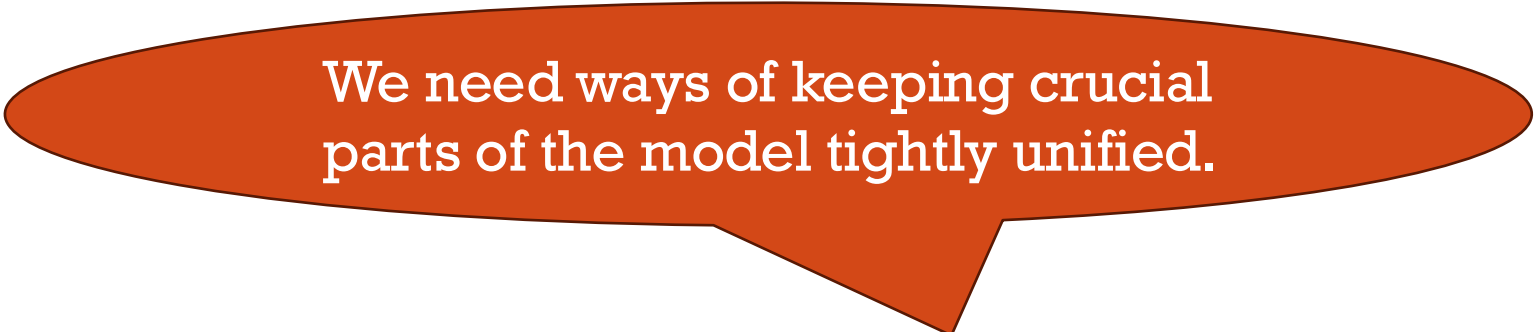
Figure 6.15. A standalone FACTORY builds AGGREGATE.



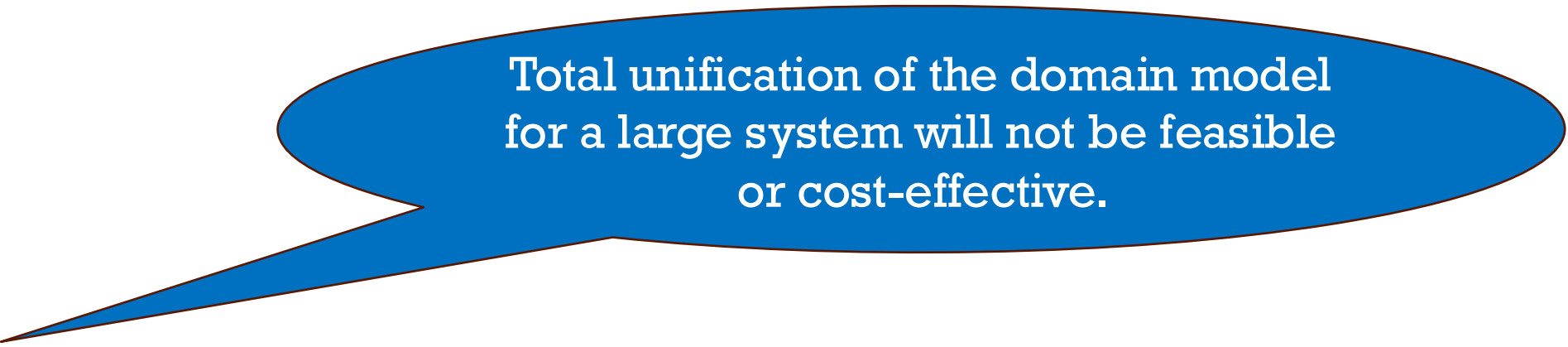
Preserving model integrity

Chapter 14 in Evan's book

HOW TO PRESERVE MODEL INTEGRITY

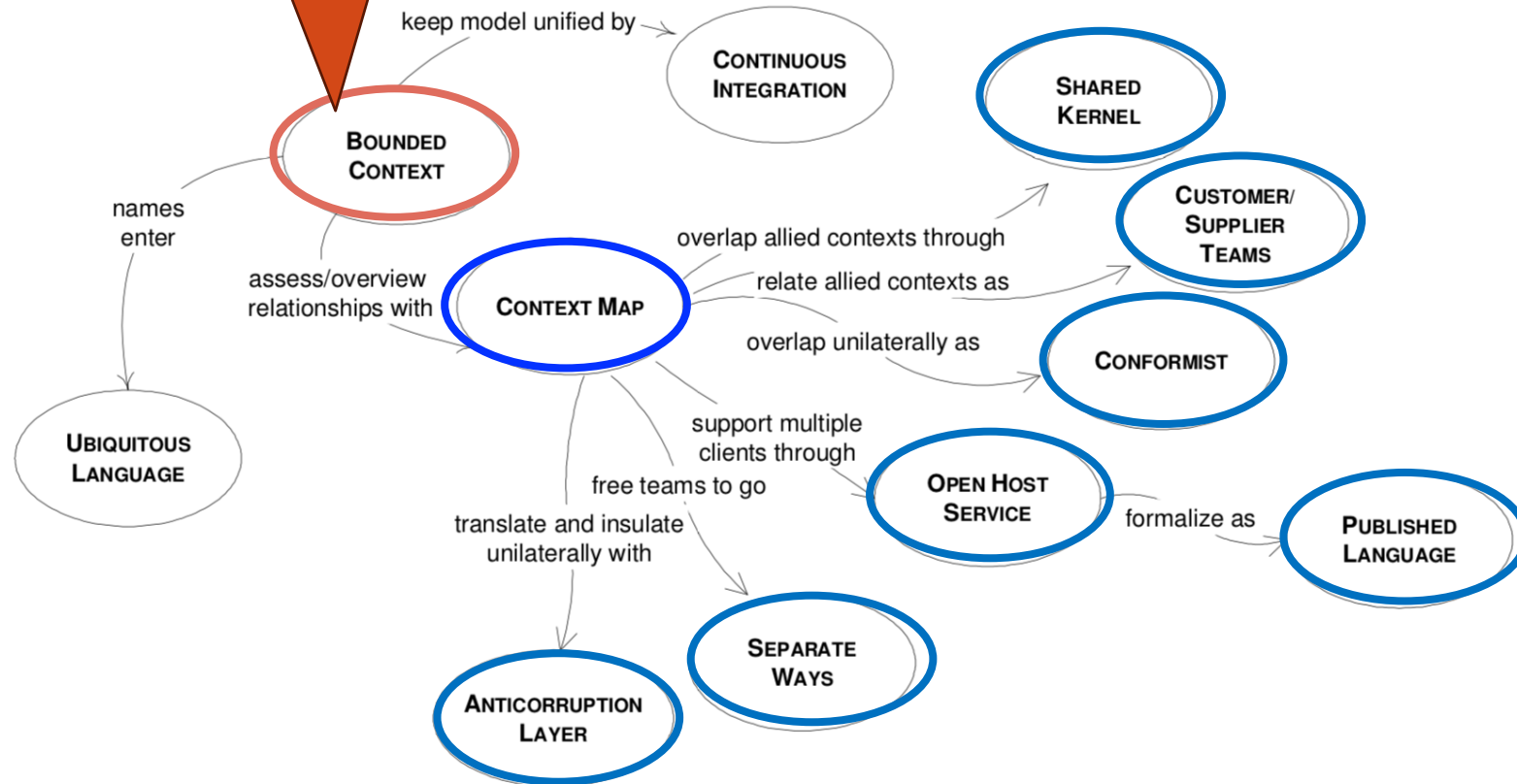


We need ways of keeping crucial parts of the model tightly unified.



Total unification of the domain model for a large system will not be feasible or cost-effective.

Defines the range of applicability of each model



**A SET OF
TECHNIQUES
USED TO
MAINTAIN
MODEL
INTEGRITY**

Bounded Context

What is the core business of Netflix?

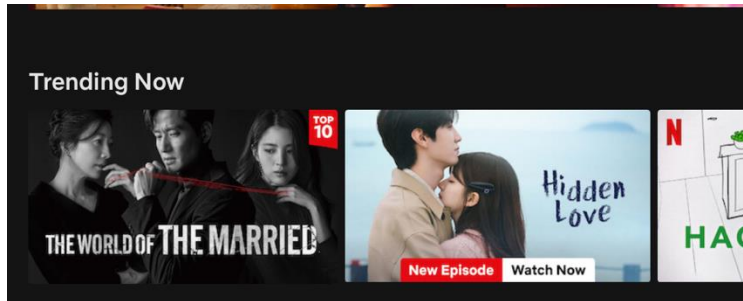
At the core of Netflix's business model is its **subscription-based model, content production and acquisition, and data-driven personalization**. Netflix, the world's leading streaming service, has revolutionized the way people consume entertainment. 27 n.w. 2567



Untaylored

<https://www.untaylored.com> › post › netflix-explaining-t...

Netflix - Explaining the Business Model and Revenue Streams



Netflix is a subscription-based streaming service that allows our members to **watch TV shows and movies on an internet-connected device**. Depending on your plan, you can also download TV shows and movies to your Android phone or tablet, iPhone, iPad, or Google Chromebook and watch without an internet connection.



Netflix Help Center

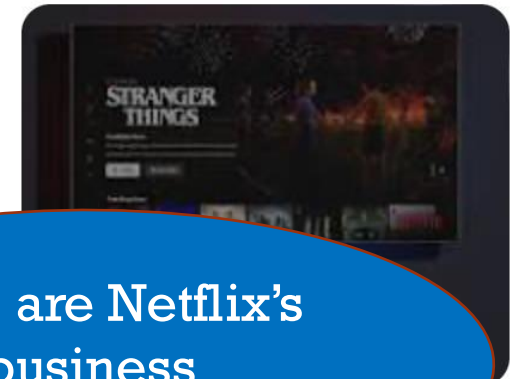
<https://help.netflix.com> › node

What is Netflix? - Netflix Help Center

What is NETFLIX' core domain?

?

What are Netflix's business capabilities?



PROBLEM IN A LARGE PROJECT

- Multiple models are in play on any large project.
- When code based on distinct models is combined, software becomes buggy, unreliable, and difficult to understand.
- Communication among team members becomes confused.
- It is often unclear in what context a model should not be applied.

THE PROBLEM, ILLUSTRATED

- Imagine one large airline system built by several teams. Each team is really working with a *different* idea of what a "Flight" or "Passenger" even means, but everyone's using the same words:
- **Booking team** thinks of a Flight as something customers search, select seats on, and pay for.
- **Operations team** thinks of a Flight as an aircraft, crew assignment, gate, and departure/arrival schedule that must comply with safety regulations.
- **Loyalty/Rewards team** thinks of a Flight mainly as a *point-earning event* tied to a Passenger's membership tier.
- **Check-in/Airport team** thinks of a Passenger as someone who needs a boarding pass, baggage tag, and security clearance — attributes the Booking team's model doesn't even have.

THE PROBLEM, ILLUSTRATED

- Imagine one large airline system that is *really working with a different set of even means*, but everyone has their own perspective.
- **Booking team** thinks of a model that can select seats on, and pay for tickets.
- **Operations team** thinks of a model that can gate, and departure/arrival times, and regular flights.
- **Marketing team** thinks of a model that can mainly as a *point-earning* program.
- **Customer service team** thinks of a model that can be as someone who can be searched for security clearance — attributes the Booking team's model doesn't even have.

If all these teams try to share **one single Flight class and one single Passenger class**, that model becomes bloated with fields and logic that only matter to *some* teams, communication breaks down

Flight.status mean 'boarding' or 'searchable for booking'

TO MANAGE A LARGE PROJECT

- Explicitly **define the context** within which a model applies.
- Explicitly **set boundaries** in terms of team organization, usage within specific parts of the application, and physical manifestations such as code bases and database schemas.
- **Keep the model strictly consistent within these bounds**, but don't be distracted or confused by issues outside.

BOUNDED CONTEXTS (AIR TRAVEL BOOKING SYSTEM)

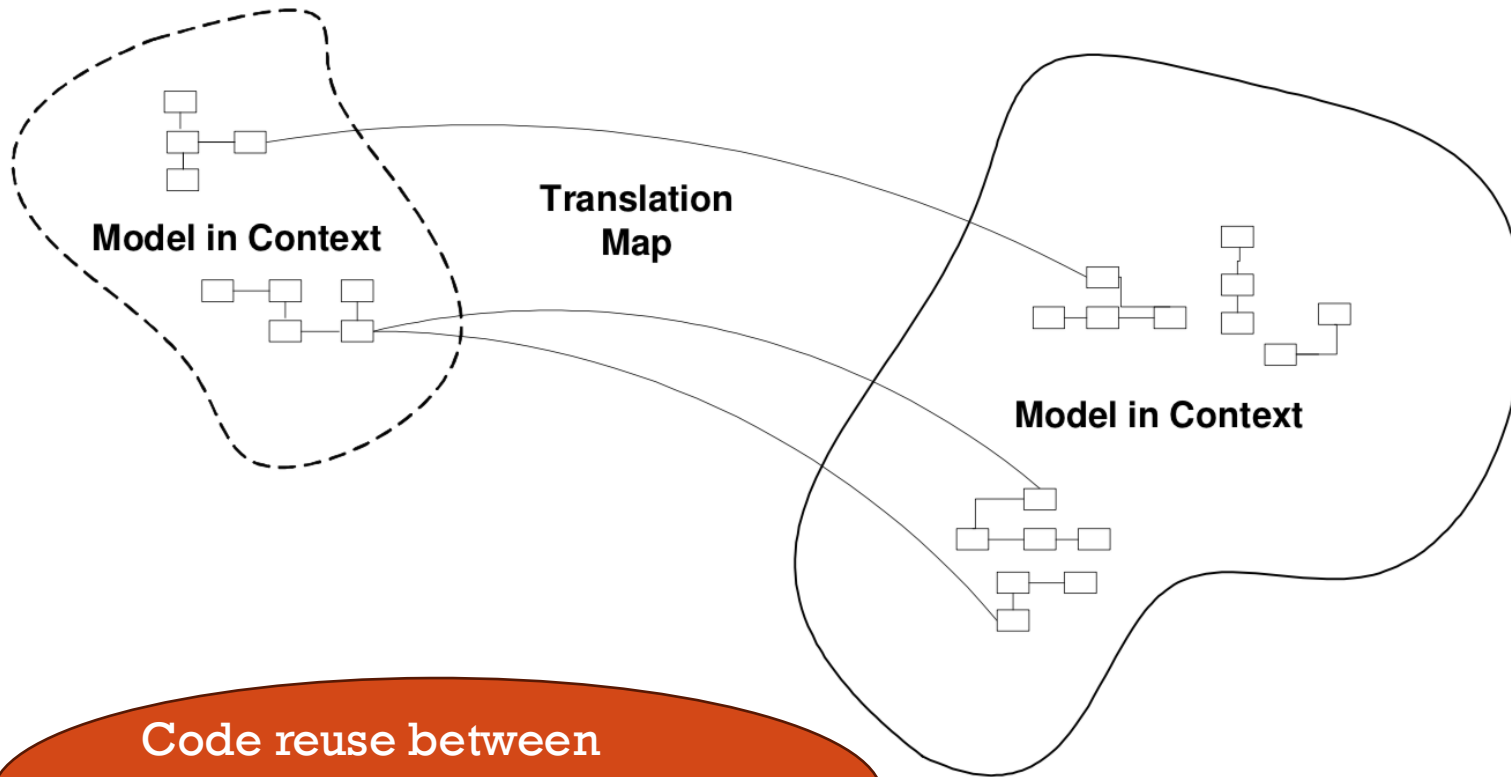
Bounded Context	Its own Flight model	Its own Passenger model	Owning team
Booking Context	Route, price, available seats, fare class	Contact info, payment method, booking history	Booking team
Operations/Scheduling Context	Aircraft assignment, crew roster, gate, maintenance status	(Passengers barely modeled here — just a headcount)	Ops team
Check-in/Boarding Context	Departure gate, boarding time, boarding status	Boarding pass, seat number, ID verification, baggage tag	Airport ops team
Loyalty Context	Just a reference/ID — "which flight earned these points"	Membership tier, points balance, points earned	Loyalty team

Context Map

A CONTEXT MAP is in the overlap between project management & software design.

CONTEXT MAP

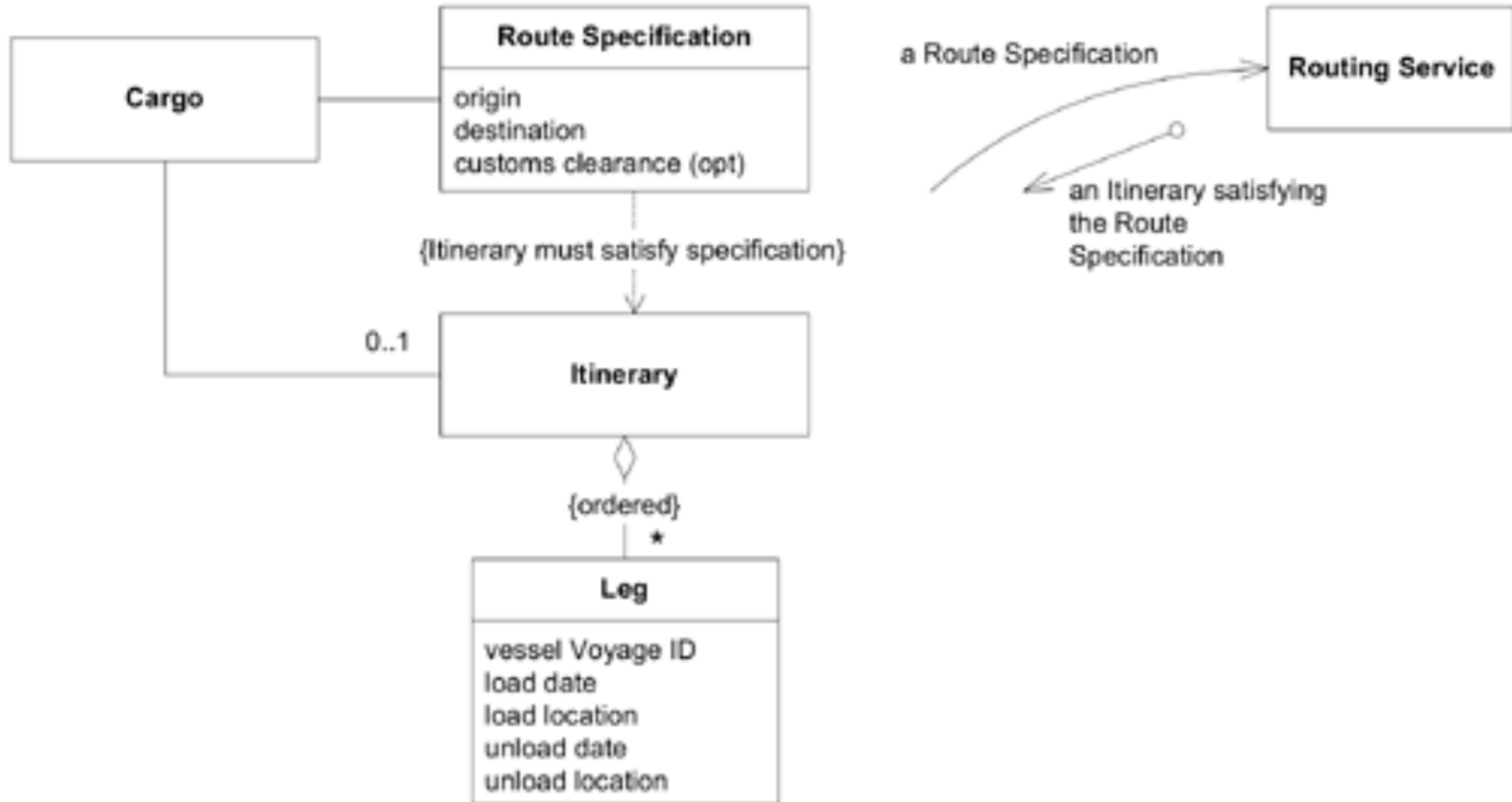
- A Context Map is a document which outlines the different Bounded Contexts and relationships between them.



Code reuse between
BOUNDED CONTEXTS is a
hazard to be avoided.

WHY?

TWO CONTEXTS IN A SHIPPING APPLICATION



TWO CONTEXTS IN A SHIPPING APPLICATION (CONT.)

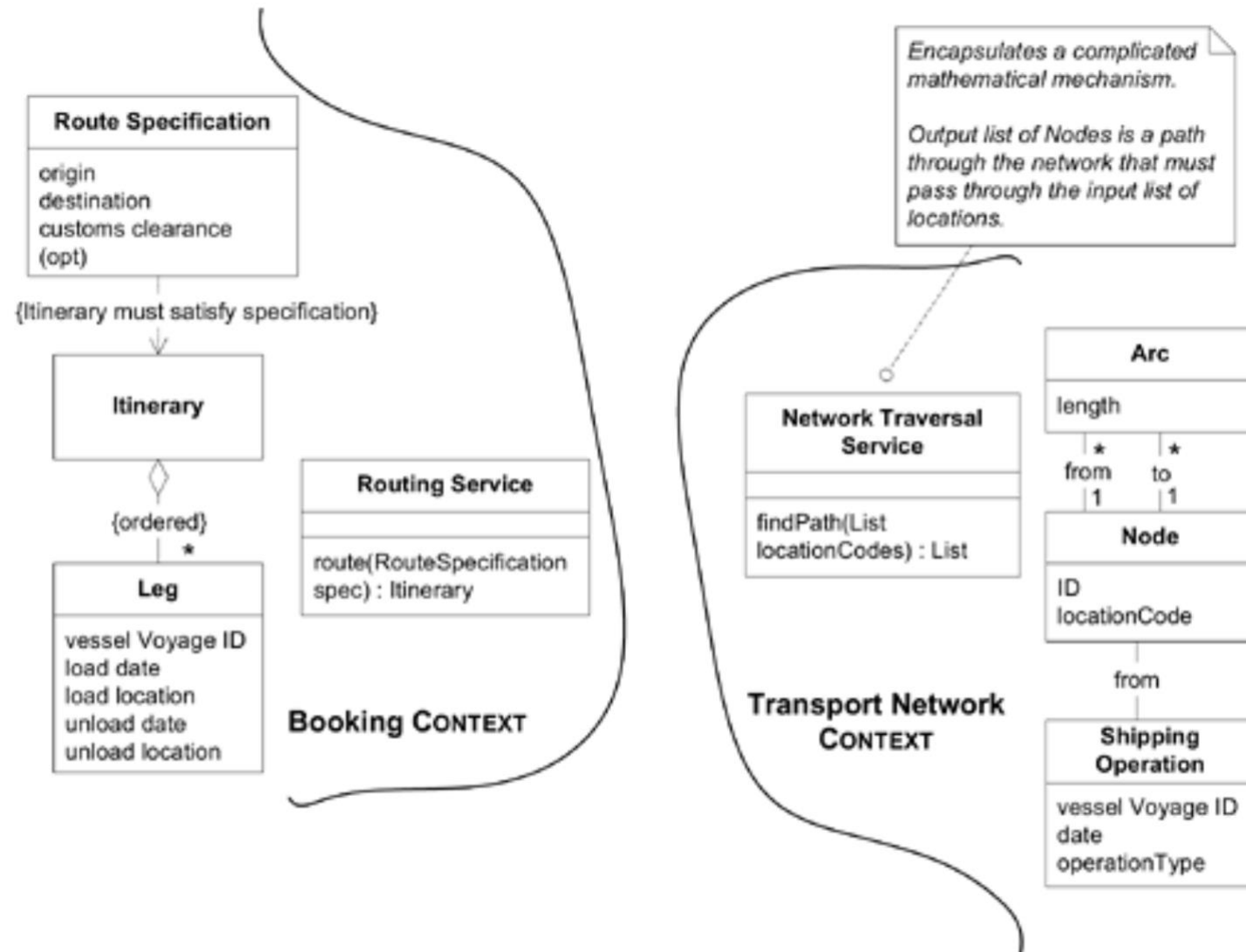
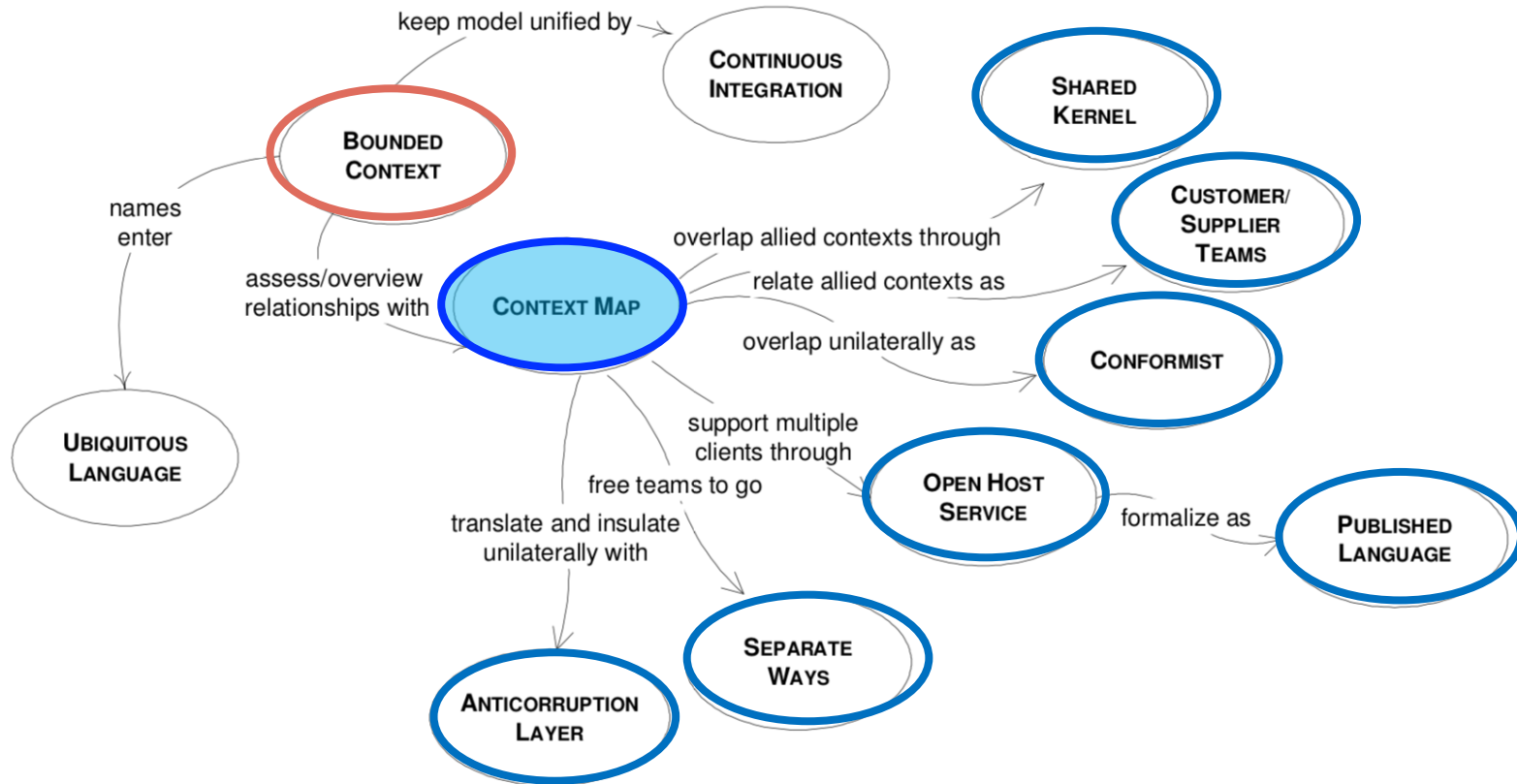


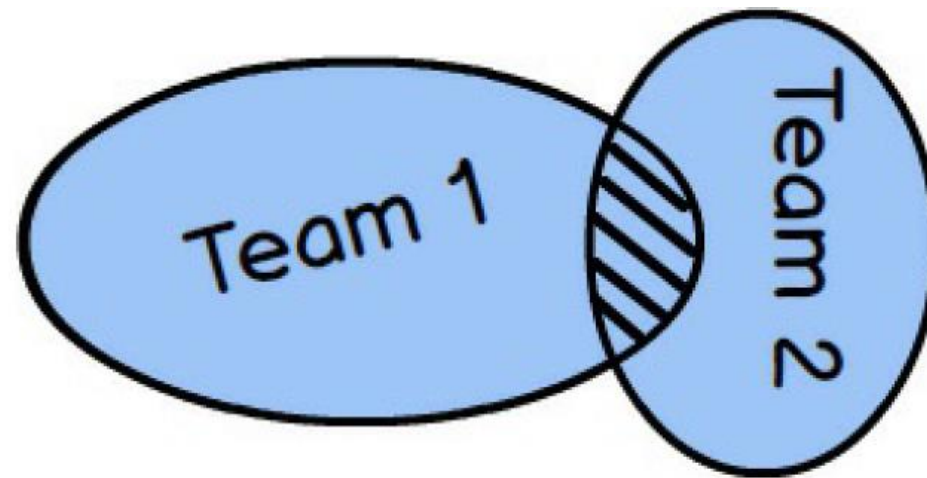
Figure 14.3. Two BOUNDED CONTEXTS formed to allow efficient routing algorithms to be applied



A SET OF TECHNIQUES USED TO MAINTAIN MODEL INTEGRITY

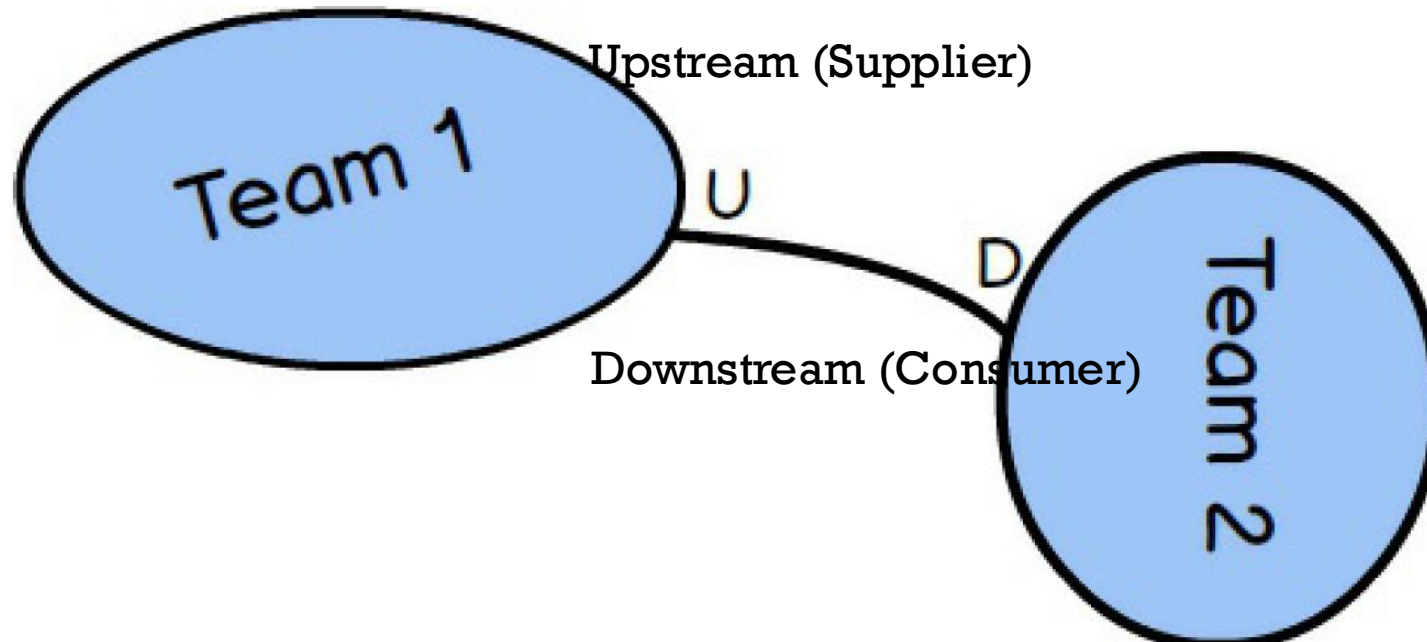
Avram A., Marinescu F., Domain-Driven Design Quickly: A summary of Eric Evans' Domain Driven Design
<https://www.lulu.com/content/325231?page=1&pageSize=4>

SHARED KERNEL



The teams must agree on what model elements they are to share.

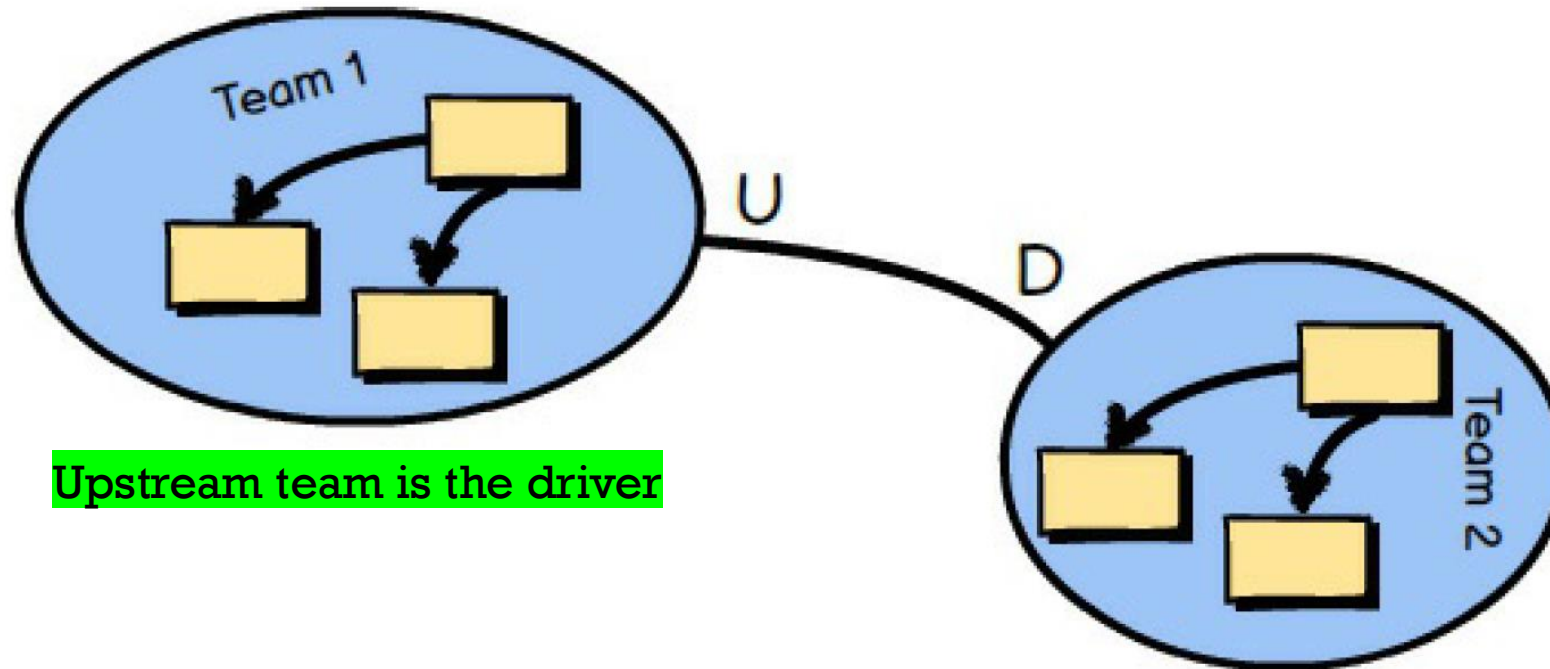
CUSTOMER-SUPPLIER



Downstream team is the driver

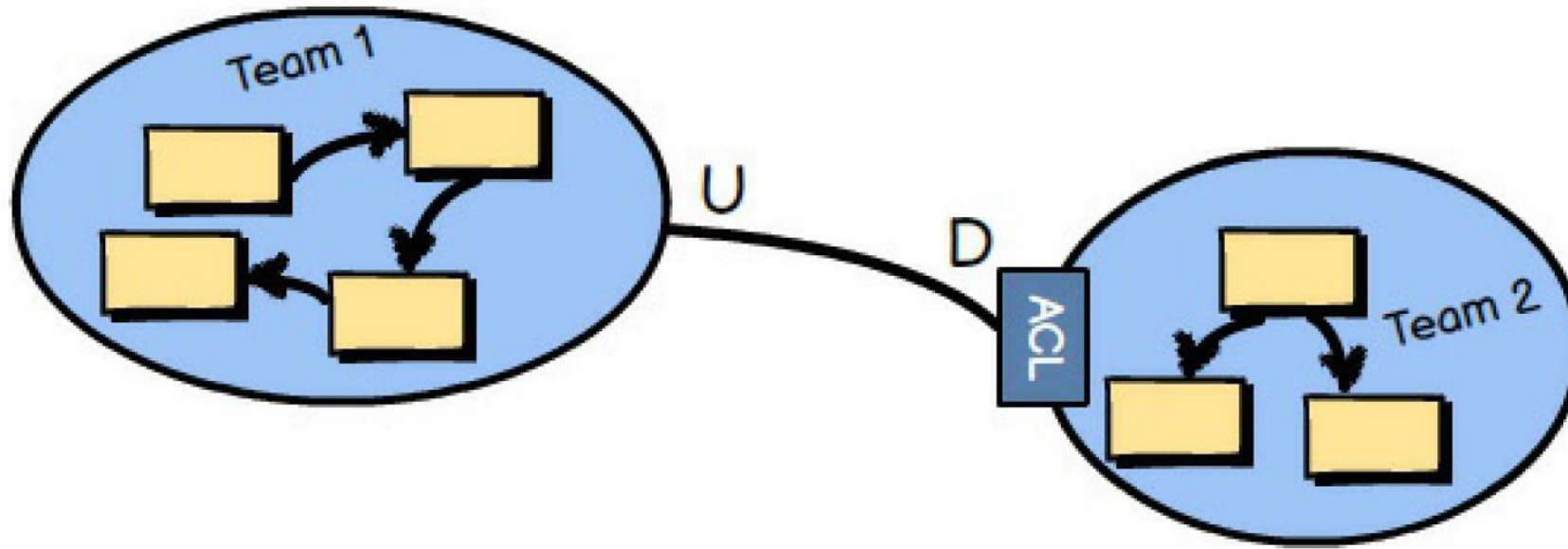
The *Supplier* must provide what the *Consumer* needs.

CONFORMIST



For various reasons the downstream team cannot sustain an effort to translate the Ubiquitous Language of the upstream model to fit its specific needs, so the team conforms to the upstream model as is.

ANTICORRUPTION LAYER (ACL)



An Anticorruption Layer is the **most defensive** Context Mapping relationship. The layer isolates the downstream model from the upstream model and translates between the two.

ANTICORRUPTION LAYER (ACL) (CONT.)

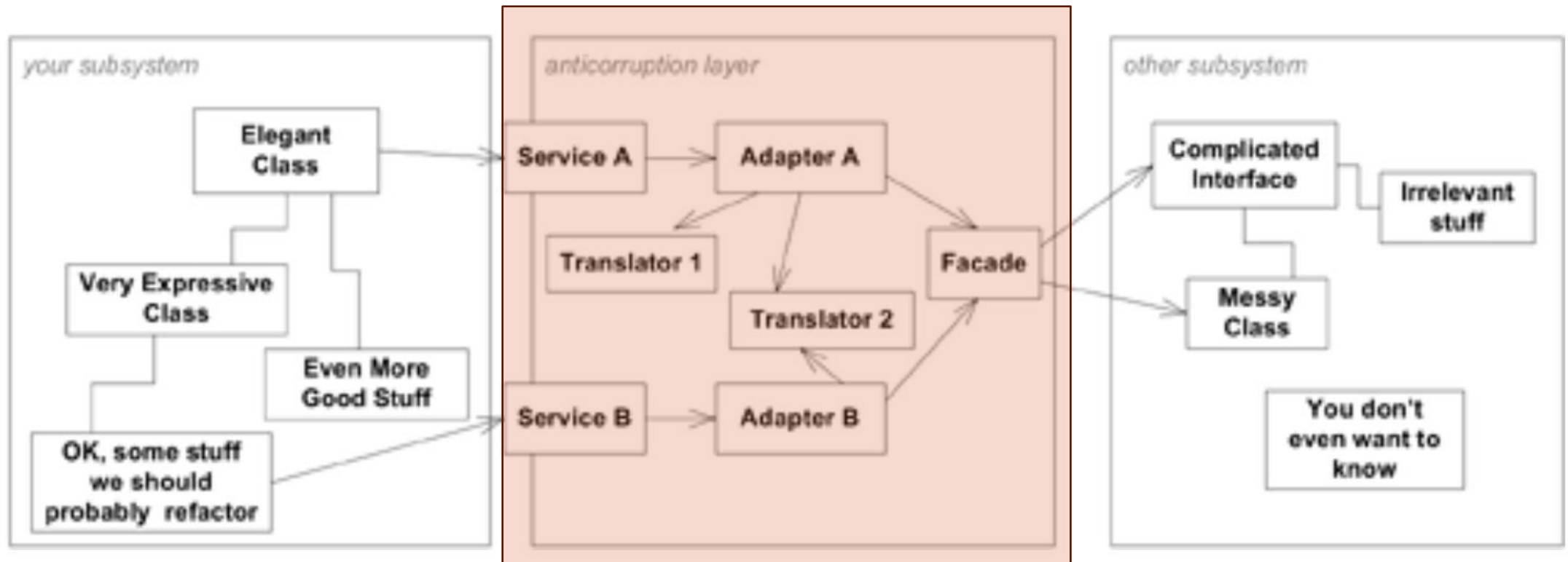
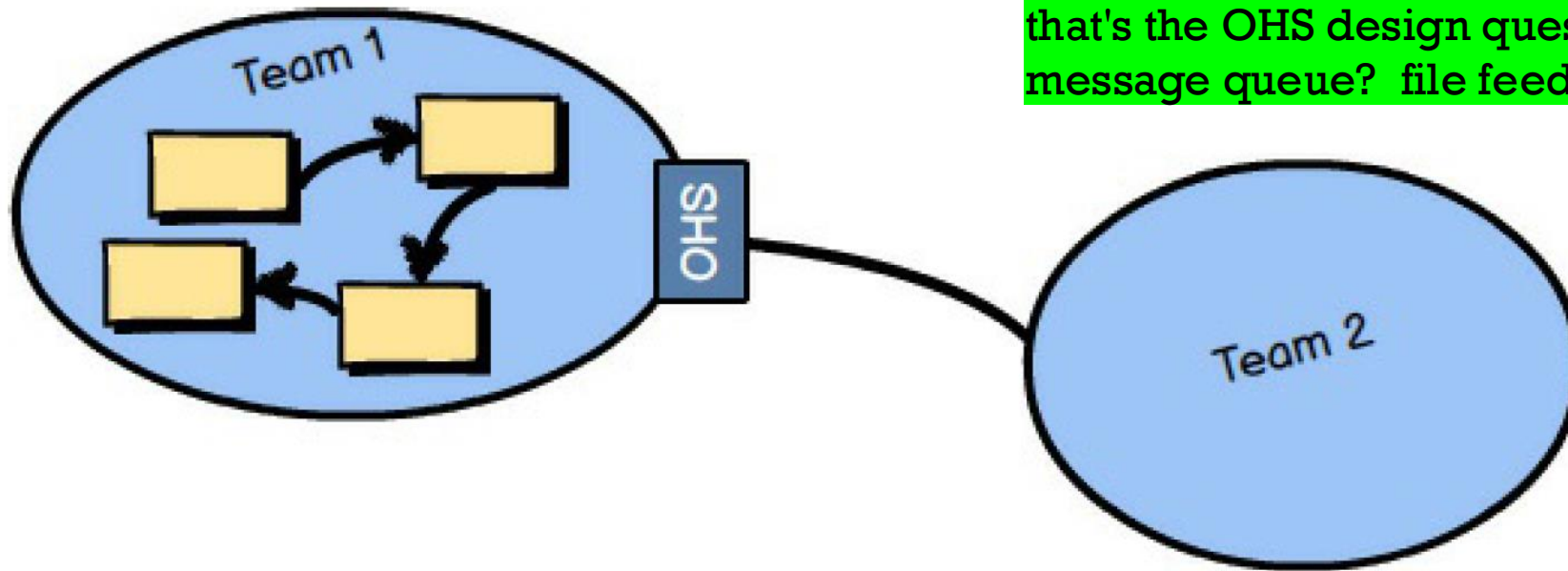


Figure 14.8. The structure of an ANTICORRUPTION LAYER

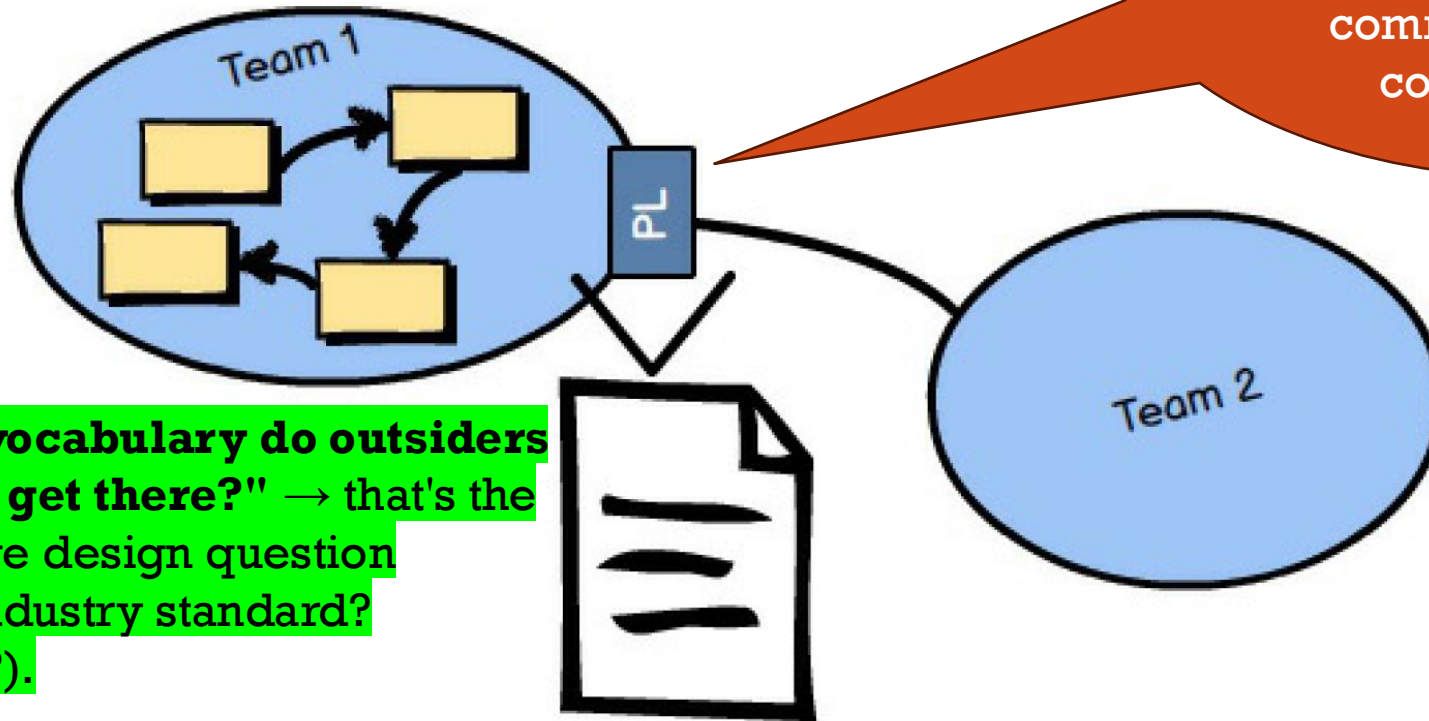
OPEN HOST SERVICE



Ask "how do outsiders reach my context?" → that's the OHS design question (REST API? gRPC? message queue? file feed?).

An Open Host Service defines a **standardized protocol or interface** that gives access to your Bounded Context as a set of services. OHS is the door.

PUBLISHED LANGUAGE



Ask "what shape/vocabulary do outsiders receive when they get there?" → that's the Published Language design question (custom schema? industry standard? versioned contract?).

Such a Published Language can be defined with XML Schema, JSON Schema, or a more optimal wire format, such as Protobuf or Avro.

MAPPING BACK TO THE OHS VS PUBLISHED LANGUAGE DISTINCTION

Concept

gRPC/Protobuf equivalent

Open Host Service (the door/mechanism)

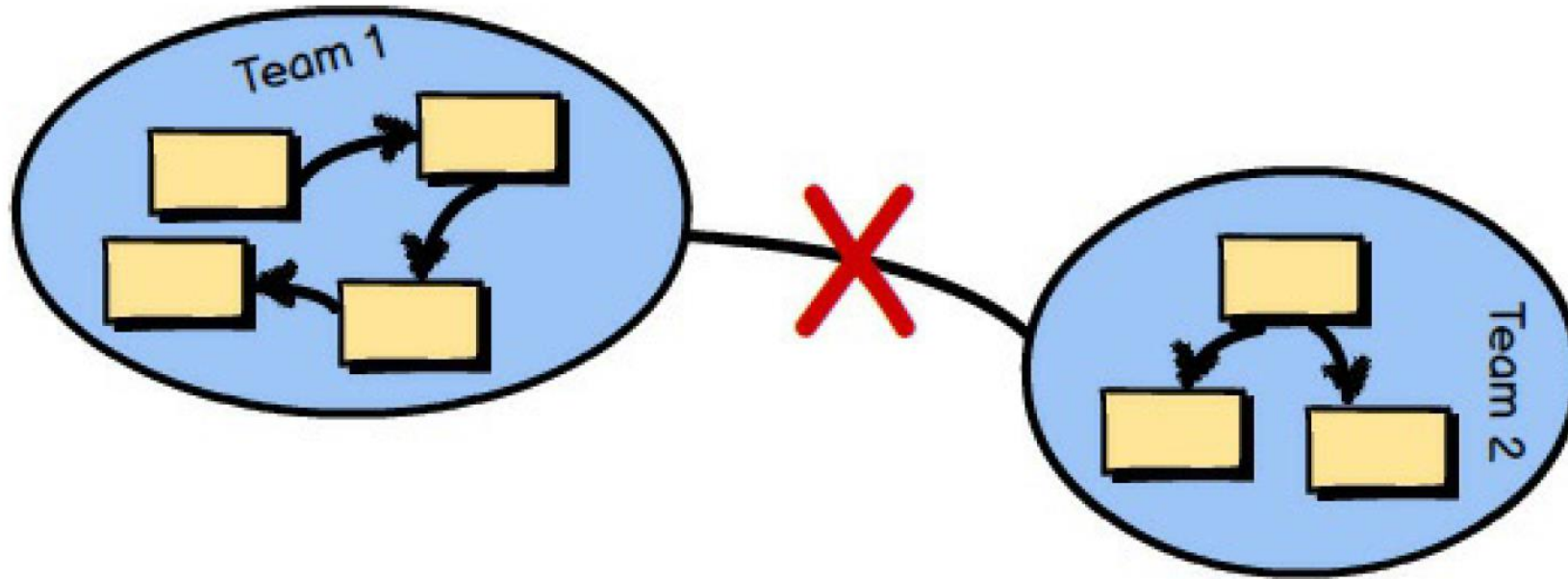
The gRPC service itself — `FlightService`, its endpoint, its transport (HTTP/2), how clients connect

Published Language (the vocabulary)

The `.proto` schema — `FlightInfoRequest/FlightInfoResponse` **message definitions**

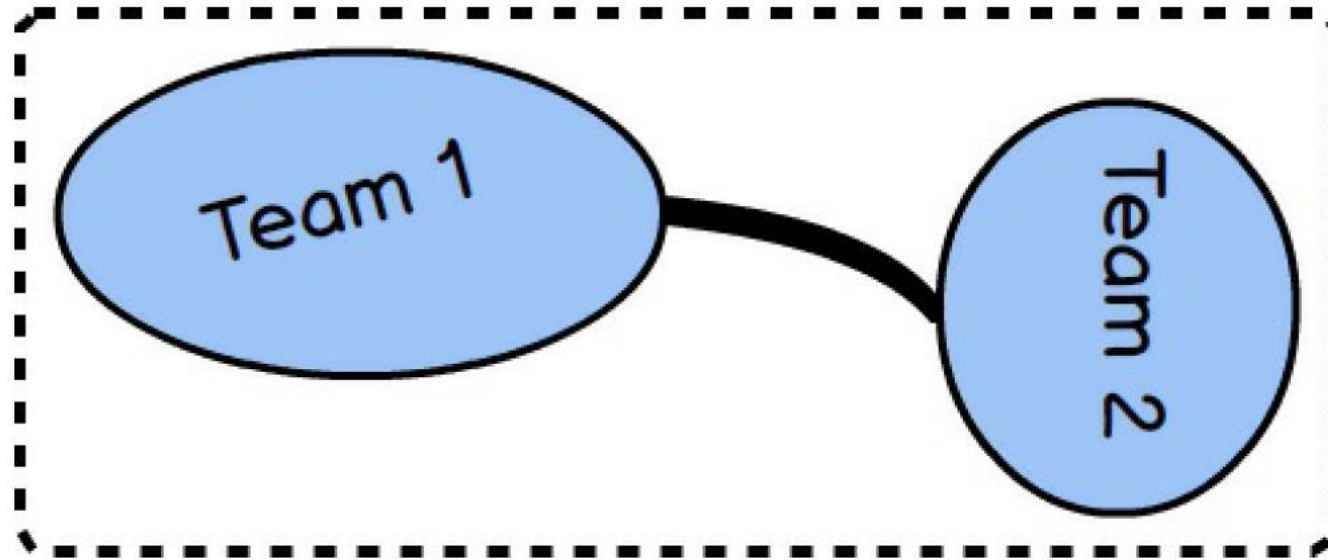
SEPARATE WAYS

Allowing developers to find simple, specialized solutions within this small scope.



In this case produce your own specialized solution in your Bounded Context and forget integrating for this special case.

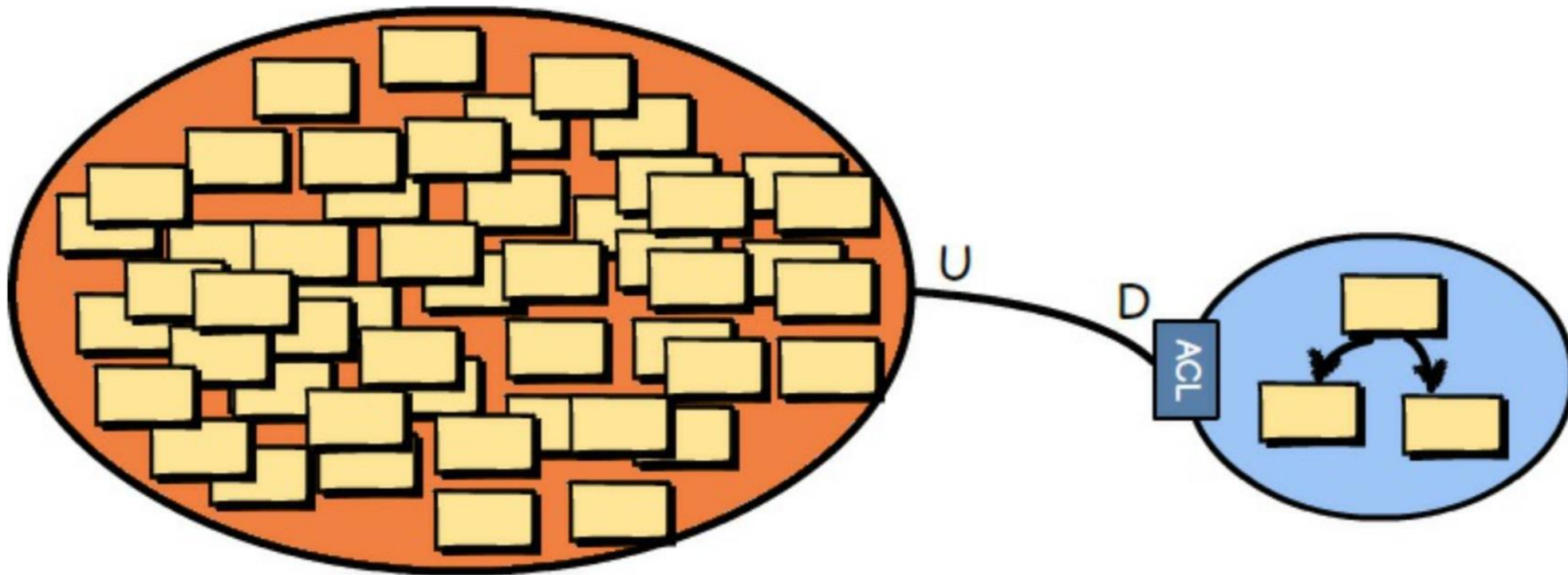
PARTNERSHIP (INTRODUCED SINCE 2004)



Each team is responsible for one Bounded Context.

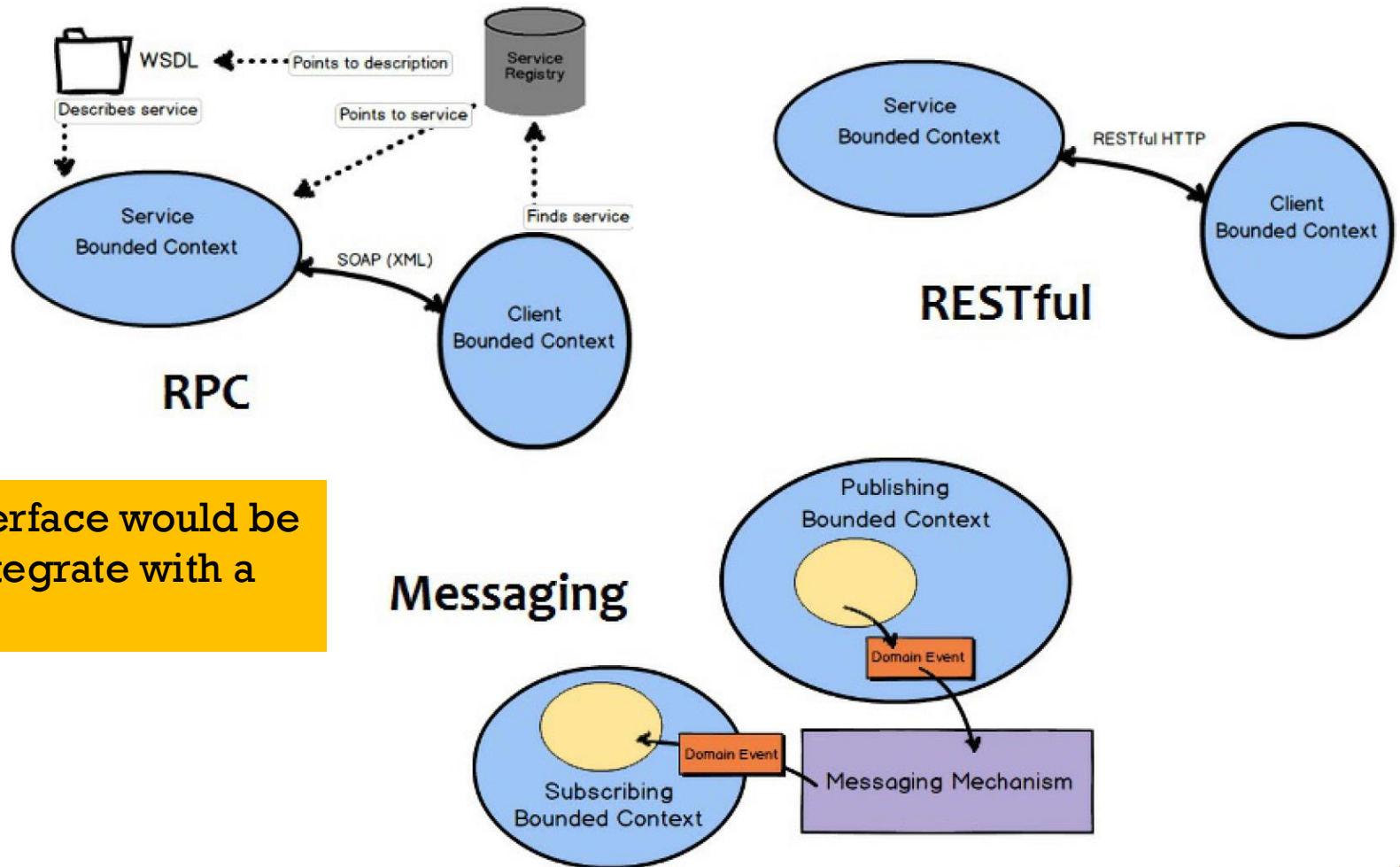
When teams in two contexts will succeed or fail together, a cooperative relationship often emerges.

BIG BALL OF MUD (INTRODUCED SINCE 2004)

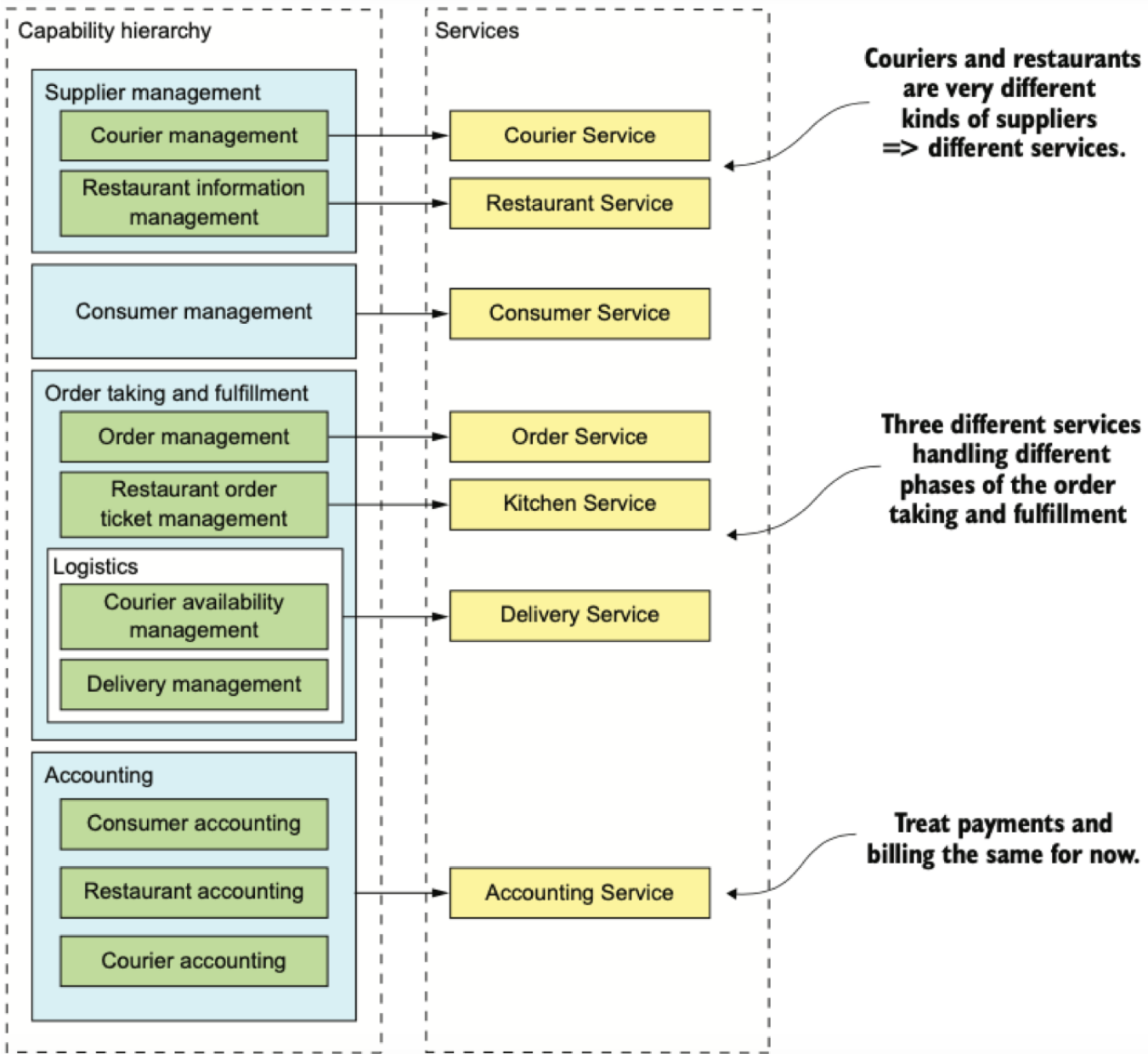


Avoid to create big ball of mud, and if need to use it, try ACL.

MAKING GOOD USE OF CONTEXT MAPPING

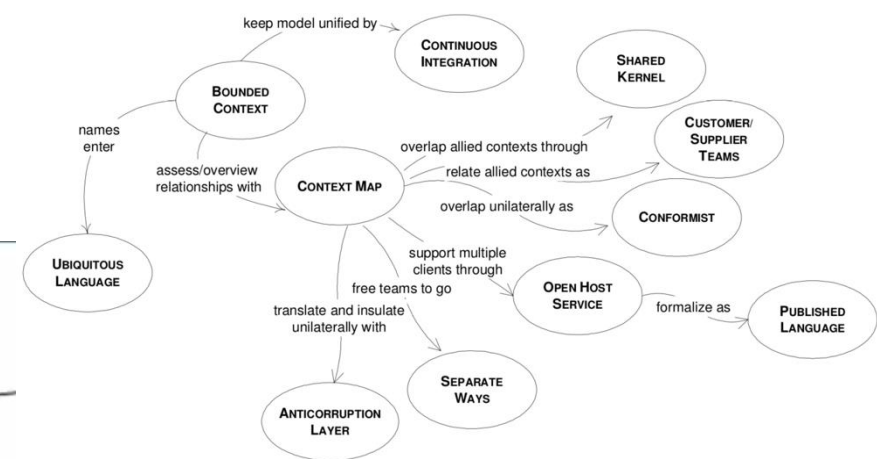


What a specific kind of interface would be supplied to allow you to integrate with a given *Bounded Context*?

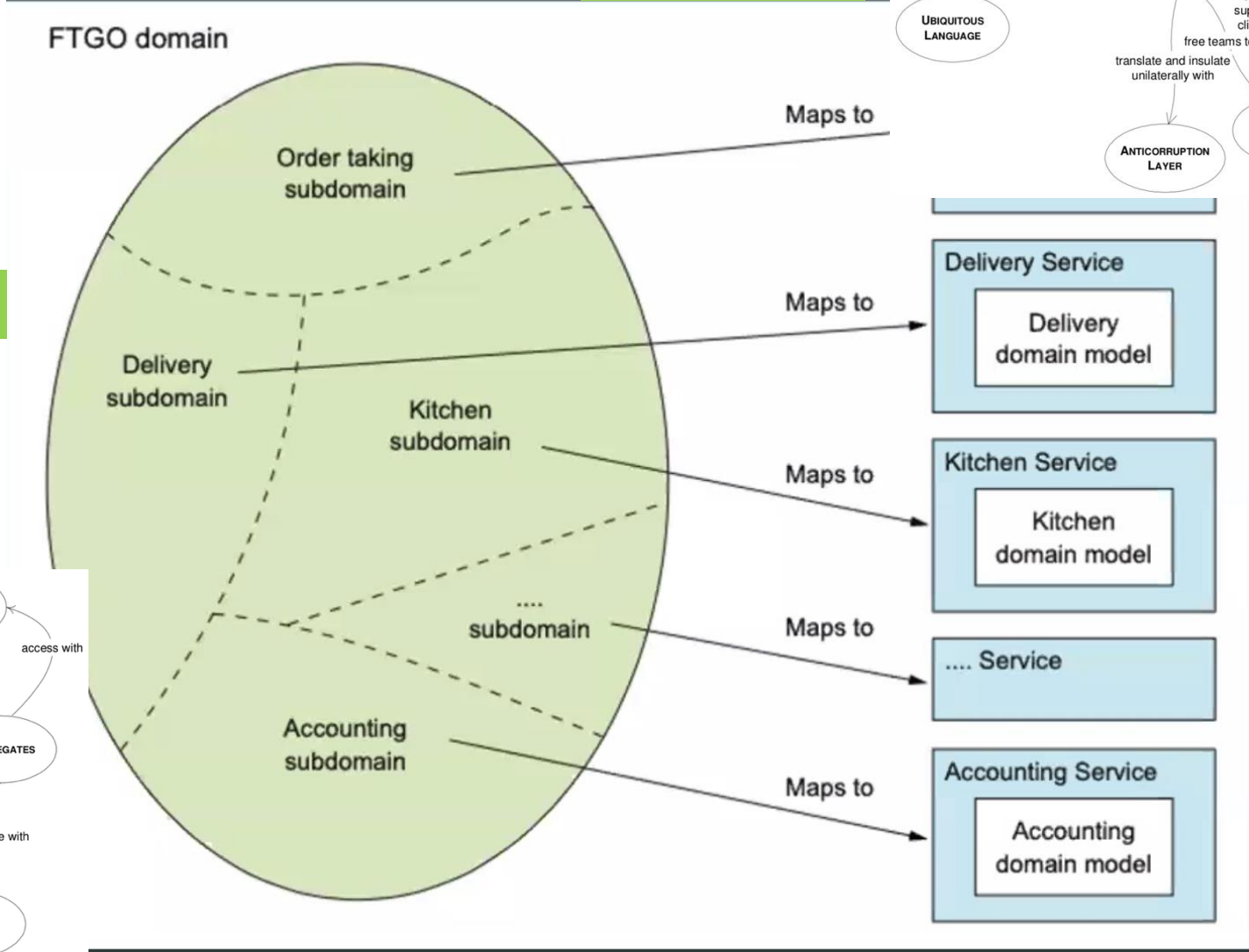
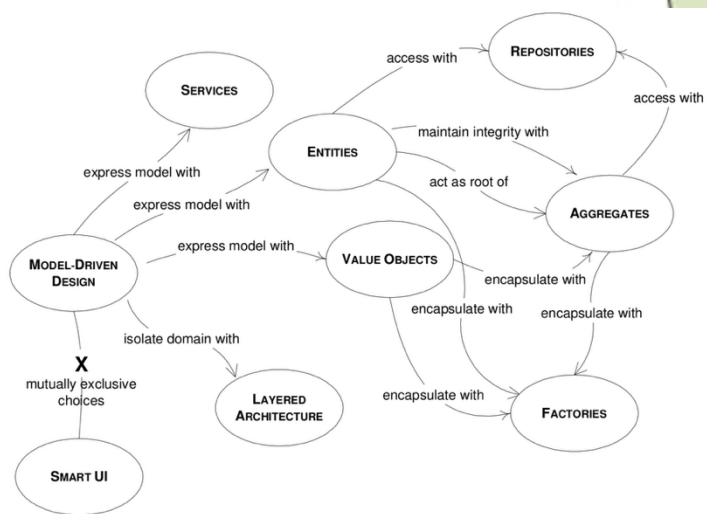


Mapping FTGO business capabilities to services

Context map

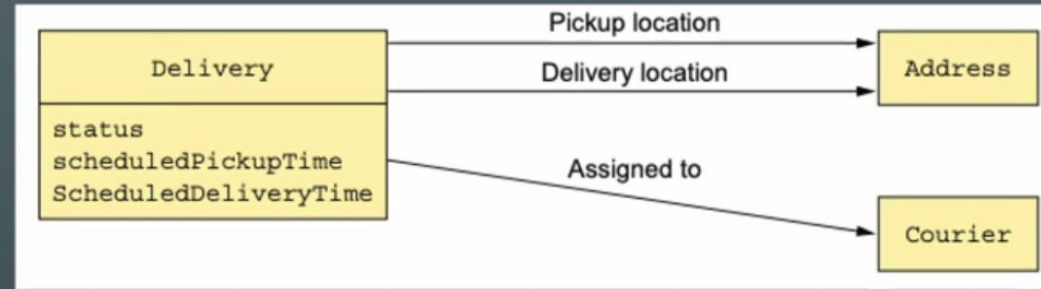


Bounded context



Applying DDD is a solution.

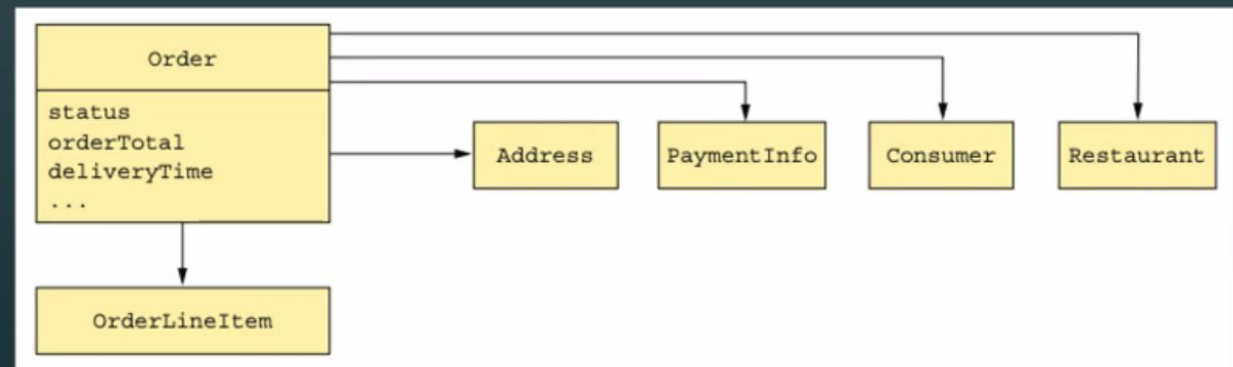
- A much better approach is to apply DDD.
- Treat each service as a separate sub- domain with its own domain model.
- The Order class in each domain model represents different aspects of the same Order business entity



The *Delivery Service* domain model



The *Kitchen Service* domain model



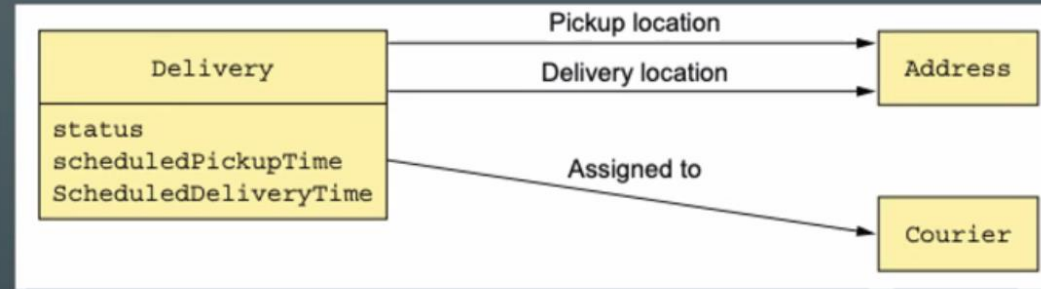
The *Order Service* domain model

- 1) What is bounded context?
- 2) Within a bounded context, what is Entities/ Value Object?
- 3) What is Aggregates?
- 4) What is repositories?
- 5) What is Services?
- 6) What is Factories?

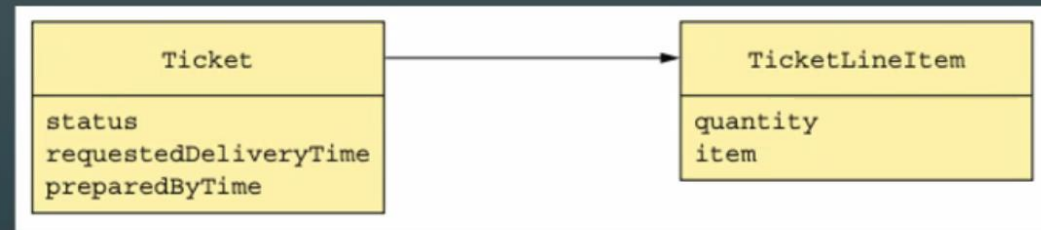
- A much better approach is to apply DDD.

1) What is context map?
What types of context map will be?

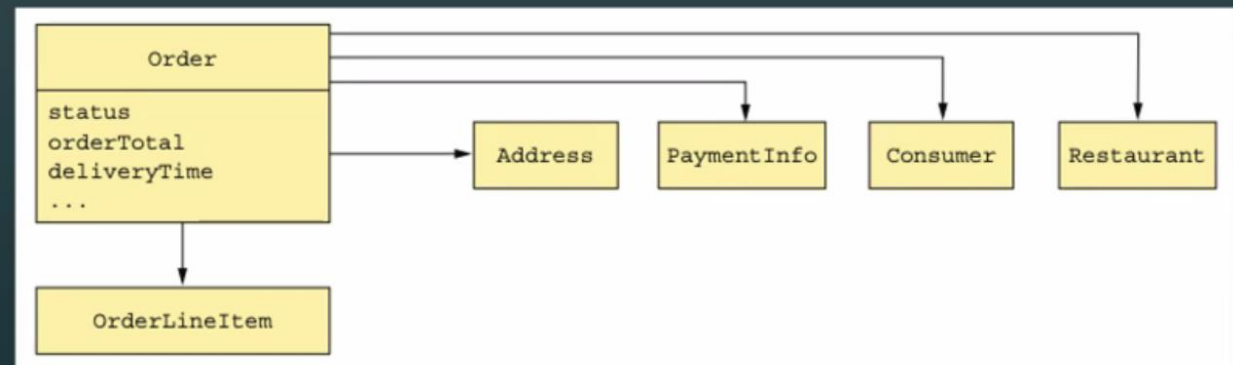
Order business entity



The *Delivery Service* domain model



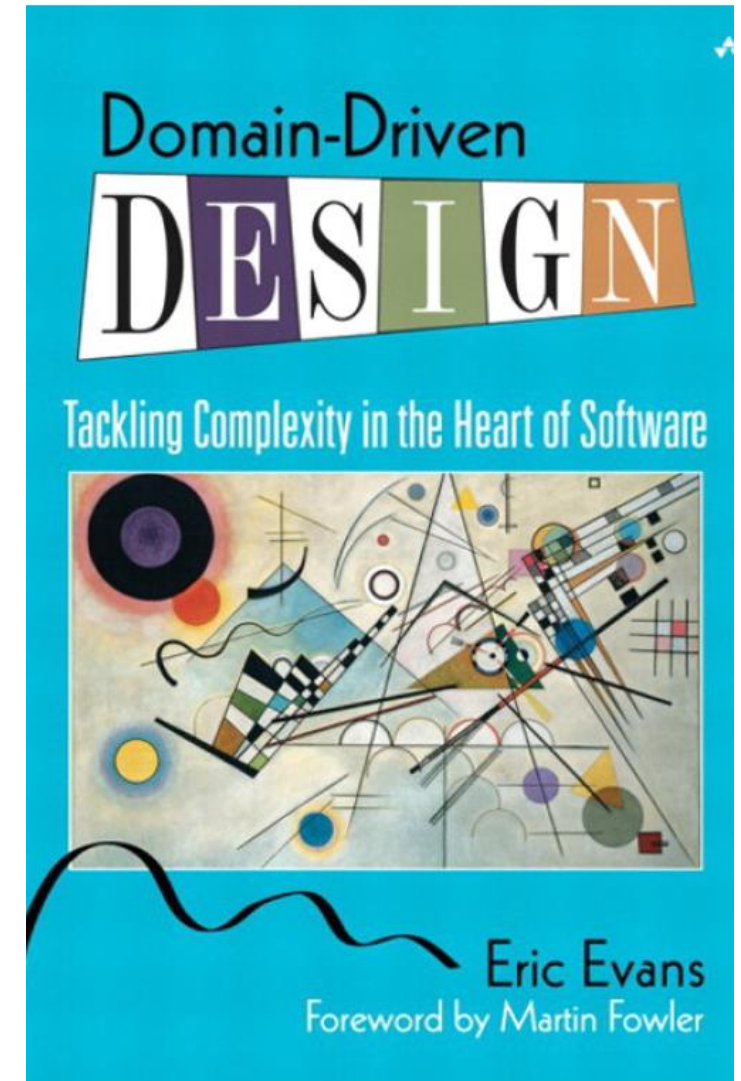
The *Kitchen Service* domain model



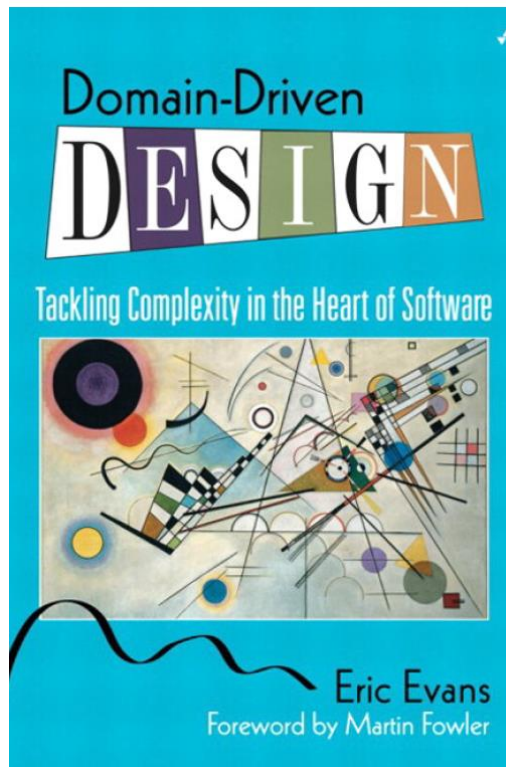
The *Order Service* domain model

REFERENCES

- Avram A., Marinescu F., Domain-Driven Design Quickly: A summary of Eric Evans' Domain Driven Design
<https://www.lulu.com/content/325231?page=1&pageSize=4>
- Vernon, V. Domain-Driven Design Distilled (2016). Chapter 4: Strategic Design with Context Mapping. Pearson Education, Inc
- <https://github.com/ernesen/DDD>
- Evans, E. Domain-Driven Design: Tackling Complexity in the Heart of Software, 2003.



TAKE HOME YOUTUBE VIDEO



Bounded Contexts - Eric Evans - DDD Europe 2020

20,878 views • Oct 2, 2020

LIKE DISLIKE SHARE SAVE ...

<https://www.youtube.com/watch?v=am-HXycfalo>

TAKE HOME YOUTUBE VIDEO



Alberto Brandolini

Event Storming

**DOMAIN DRIVEN
DESIGN EUROPE**

January 29 - February 1 - 2019
dddeurope.com

Xebia

EVENT STORE

#EventStorming

Event Storming - Alberto Brandolini - DDD Europe 2019

<https://www.youtube.com/watch?v=mLXQIYEwK24>

Introducing EventStorming

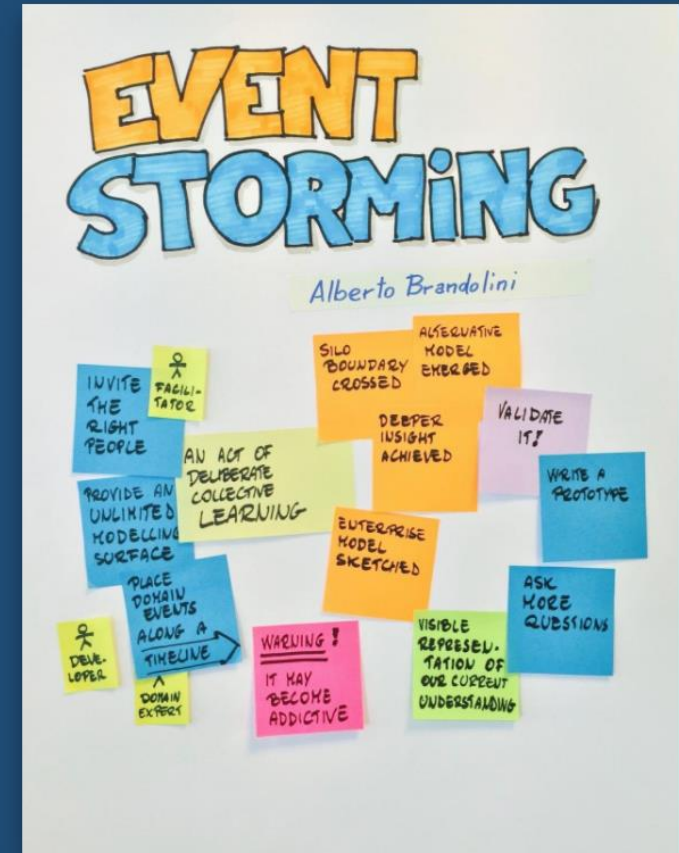
An act of deliberate collective learning

The book is available on Leanpub (PDF, EPub and Mobi).

It's not finished, but there's already enough content to help explorers and practitioners.

[Read the book >](#)

Paper version will follow



IN CLASS GROUP ASSIGNMENT

Core business
capabilities

ทำโปรเจกอะไร

- มี domain อะไรบ้าง / มี Event อะไรบ้าง (1-2 events)
- Ubiquitous language
- ยกตัวอย่าง Entity / value objects / service ใน 1 domain
- ยกตัวอย่าง context map มา 1 context map พร้อมเหตุผล

TAKE HOME MESSAGE

- “DDD is less about software design patterns and more about problem solving through collaboration.
- Evans presents techniques to use software design patterns to enable models created by the development team and business experts to be implemented using the UL.
- However, without the practices of analysis, and collaboration, the coding implementation really means very little on its own
- DDD is not code centric; its purpose is not to make elegant code. Software is merely an artifact of DDD.

“

TAKE HOME MESSAGE

- **Strategic design** focuses on defining the bounded contexts, ubiquitous languages, and context maps
- **Tactical design** is a set of technical resources used in constructing domain model in a bounded context.